Unidade IV

7 TÓPICOS ESPECIAIS EM C#

No desenvolvimento de software, a linguagem C# tem se destacado como ferramenta robusta, com uma variedade de recursos que otimizam a codificação. Esses recursos, embora distintos à primeira vista, frequentemente se inter-relacionam, refletindo a interconexão e a coesão intrínseca da própria linguagem.

As expressões lambda e o LINQ são dois desses recursos que revelam uma profunda integração no que diz respeito à manipulação de dados em C#. Expressões lambda são representações sintáticas concisas de funções anônimas, permitindo operações mais fluidas e legíveis. O LINQ, por sua vez, fornece mecanismos para consultar e transformar coleções de dados em C#, integrando-se naturalmente com expressões lambda para criar consultas altamente expressivas sem a necessidade de linguagens de consulta externas. A gestão eficiente de recursos e o diagnóstico de problemas também têm sua intersecção no C#.

Disposal e Garbage Collection são ferramentas centrais para gerenciar memória. Enquanto o Garbage Collection se preocupa com a limpeza automática de objetos não referenciados, liberando memória e otimizando a performance, o Disposal garante a liberação adequada de recursos não gerenciados. Por outro lado, debugging e tracing atuam em conjunto para monitorar e diagnosticar a execução do programa, seja identificando falhas específicas ou rastreando o comportamento ao longo do tempo.

Ao lidar com a evolução constante da programação, o C# introduziu recursos como o pattern matching, mecanismo que permite verificar se um valor corresponde a determinado padrão e, em seguida, extrair informações desse valor de forma concisa. Em sintonia com essa busca por eficiência, temos a implementação de threading, tasks e funções assíncronas. Enquanto threading abre caminho para operações simultâneas, maximizando a utilização da CPU, as tasks e funções assíncronas simplificam a execução de tarefas prolongadas, tornando o código não apenas mais eficiente, mas também mais responsivo.

Nesse panorama, é evidente que os recursos especiais do C# não são meramente adições isoladas à linguagem. Eles coexistem, interagem e se complementam, formando um ecossistema coeso, que capacita desenvolvedores a criar soluções sofisticadas e de alta qualidade. Explorar e compreender essas facetas enriquece a experiência de codificação, conduzindo um desenvolvimento de software mais integrado.

7.1 Expressões lambda

Desde sua introdução no C# 3.0, as expressões lambda se tornaram um elemento fundamental da linguagem, permitindo uma forma mais concisa e legível de representar funções anônimas. Elas representam uma evolução das delegações anônimas, mas com uma sintaxe muito mais refinada e expressiva.

Uma expressão lambda é, em sua essência, uma forma compacta de definir um método. Pelo operador "**=>**", ela separa os parâmetros da função de seu corpo. Por exemplo, a expressão lambda (x, y) => x + y define uma função que pega dois parâmetros e retorna sua soma. Ao usar essa expressão, programadores podem criar funções sem precisar nomeá-las, o que é especialmente útil em operações curtas, usadas como argumento para métodos de ordem superior.

Um dos principais lugares onde as expressões lambda são usadas é com Language Integrated Query (LINQ), uma série de extensões de métodos que permitem manipular coleções de dados de maneira declarativa (assunto mais detalhado no tópico 7.2). Mas além da sintaxe concisa, o que realmente torna as expressões lambda poderosas é o fato de elas capturarem o contexto em que são definidas. Isso significa que uma expressão lambda pode acessar variáveis do seu escopo circundante, permitindo a criação de closures. Essa capacidade de capturar o estado torna as expressões lambda extremamente flexíveis, permitindo aos programadores criar funções que se comportam de acordo com o contexto em que foram criadas. Contudo, apesar das vantagens, é importante usar expressões lambda com discernimento.

```
1.  using System;
2.  using System.Collections.Generic;
3.
4.  public class Programa
5.  {
6.    public delegate bool VerificaDeus(string deus);
7.
8.    public static void Main()
9.    {
10.     List<string> deuses = new List<string>
11.     {
12.       "Zeus",
13.       "Apolo",
14.       "Afrodite",
15.       "Hades",
16.       "Atena",
17.       "Hermes"
18.     };
19.
20.     VerificaDeus eDeusDoOlimpo = (nome) => nome != "Hades";
21.
22.     foreach (var deus in deuses)
23.     {
24.       if (eDeusDoOlimpo(deus))
25.       {
26.         Console.WriteLine($"{deus} é um deus do Olimpo.");
27.       }
28.       else
29.       {
30.         Console.WriteLine($"{deus} não é um deus do Olimpo.");
31.       }
32.     }
33.   }
34. }
```

Figura 153 – Expressões lambda: exemplo de utilização

Em alguns casos, especialmente quando a lógica começa a se tornar mais complexa, pode ser mais apropriado definir um método nomeado ou até mesmo uma classe para representar a funcionalidade desejada. Isso facilita a leitura, o teste e a manutenção do código.

A figura 153 apresenta um exemplo de uso de expressões lambda. O programa inicia definindo uma lista de deuses, que inclui nomes como Zeus, Apolo, Afrodite e outros. Em seguida, temos a declaração de um delegado chamado VerificaDeus. Como vimos no tópico 5.2, delegados são tipos que representam métodos com parâmetros e um valor de retorno específico; no caso, o delegado representa um método que leva uma string (o nome de um deus) e retorna um valor booleano.

O benefício do lambda fica claro quando definimos a variável eDeusDoOlimpo. Em vez de criar um método separado para verificar se um deus pertence ao Olimpo, basta usar uma expressão lambda. A lógica é simples: se o nome do deus for Hades, ele não pertence ao Olimpo; caso contrário, pertence.

A parte final do código percorre a lista de deuses e utiliza a função representada pelo lambda para verificar se cada deus pertence ou não ao Olimpo. Baseado nessa verificação, imprime uma mensagem adequada para cada deus.

A grande vantagem do lambda aqui é a concisão. Em vez de precisar criar um método separado e nomeado para a verificação, podemos simplesmente definir a lógica diretamente onde ela é necessária, tornando o código mais limpo e legível. Além disso, a expressão lambda oferece uma maneira flexível de definir comportamentos que podem ser facilmente modificados ou estendidos no futuro.

7.2 LINQ: consultas e operadores

LINQ é uma característica robusta do C# e parte fundamental do .NET Framework, permitindo consultas de dados de forma mais intuitiva e segura, de modo integrado à linguagem. Essa tecnologia é usada principalmente para buscar e filtrar dados de diferentes fontes, como coleções de objetos, bancos de dados e Extensible Markup Language (XML), de maneira declarativa e consistente. Também possibilita consultas SQL-like diretamente em coleções de objetos em C#, proporcionando uma sintaxe de consulta coerente e uniforme, apresentando uma séria de operadores de consulta que permitem ações como seleção, filtragem, ordenação, agrupamento, entre outras, em diferentes fontes de dados.

Para facilitar ainda mais o desenvolvimento, a LINQ oferece suporte a consultas compostas e projetadas, viabilizando a criação de consultas complexas e versáteis. Além disso, proporciona uma verificação de tipo em tempo de compilação, tornando o código mais seguro ao reduzir erros em tempo de execução, aumentando a qualidade e a manutenibilidade do código-fonte. Com a capacidade de trabalhar com diferentes fontes de dados e oferecer uma sintaxe de consulta integrada e concisa, a linguagem torna-se ferramenta essencial para desenvolvedores .NET, possibilitando criar soluções de software mais eficientes e robustas.

Essa tecnologia tem diferentes provedores, como o LINQ to Objects (para trabalhar com coleções em memória), LINQ to SQL (para trabalhar com bancos de dados relacionais) e LINQ to XML (para manipular dados em formato XML). Cada provedor tem suas particularidades, mas todos compartilham do mesmo princípio básico de prover uma interface de consulta integrada e uniforme.

Usando LINQ, a manipulação e a consulta de dados tornam-se mais intuitivas, limpas e coesas, reduzindo a complexidade associada a operações de dados e aumentando a produtividade do desenvolvedor.

```
1.  using System;
2.  using System.Collections.Generic;
3.  using System.Linq;
4.  class Programa
5.  {
6.     static void Main()
7.     {
8.        // Criação de uma lista de deuses da mitologia grega
9.        List<Deus> deuses = new List<Deus>
10.       {
11.          new Deus { Nome = "Zeus", Dominio = "Céu", Simbolo = "Raio" },
12.          new Deus { Nome = "Poseidon", Dominio = "Mar", Simbolo =
"Tridente" },
13.          new Deus { Nome = "Hades", Dominio = "Submundo", Simbolo =
"Capacete da Invisibilidade" },
14.          new Deus { Nome = "Atena", Dominio = "Sabedoria", Simbolo =
"Coruja" },
15.          new Deus { Nome = "Apolo", Dominio = "Sol", Simbolo = "Lira" }
16.       };
17.       // Uso do LINQ para encontrar todos os deuses cujo domínio é o
céu
18.       var deusesDoCeu = from deus in deuses
19.                 where deus.Dominio == "Céu"
20.                 select deus;
21.       Console.WriteLine("Deuses do Céu:");
22.       foreach (var deus in deusesDoCeu)
23.       {
24.          Console.WriteLine(deus.Nome);
25.       }
26.       // Uso do LINQ para ordenar os deuses pelo nome
27.       var deusesOrdenados = from deus in deuses
28.                 orderby deus.Nome ascending
29.                 select deus;
30.       Console.WriteLine("\nDeuses Ordenados por Nome:");
31.       foreach (var deus in deusesOrdenados)
32.       {
33.          Console.WriteLine(deus.Nome);
34.       }
35.    }
36. }
37. class Deus
38. {
39.    public string Nome { get; set; }
40.    public string Dominio { get; set; }
41.    public string Simbolo { get; set; }
42. }
```

Figura 154 – LINQ to Objects: exemplo de utilização

A linha 37 define uma classe Deus com propriedades como Nome, Dominio e Simbolo. Na linha 9 criamos uma lista chamada deuses que armazena objetos do tipo Deus, representando diferentes deidades da mitologia grega. O primeiro uso de LINQ no código filtrou os deuses cujo domínio é o céu,

usando a expressão **where** (linha 18). A consulta é armazenada na variável deusesDoCeu, e os resultados são impressos na tela, mostrando apenas os deuses que satisfazem a condição.

O segundo uso de LINQ é ordenar os deuses pelo nome em ordem ascendente usando a cláusula order by (linha 28). A consulta ordenada é armazenada na variável deusesOrdenados, que são então impressos na tela. Ambas as consultas são exemplos do LINQ to Objects, no qual operações de consulta são realizadas em coleções de objetos em memória, nesse caso a lista de deuses.

```
1.  using System;
2.  using System.Linq;
3.  using System.Xml.Linq;
4.
5.  class Programa
6.  {
7.    static void Main()
8.    {
9.      // Criando um documento XML representando deuses da mitologia
grega.
10.     XDocument documentoMitologiaGrega = new XDocument(
11.       new XElement("Deuses",
12.         new XElement("Deus",
13.           new XElement("Nome", "Zeus"),
14.           new XElement("Dominio", "Céu"),
15.           new XElement("Simbolo", "Raio")),
16.         new XElement("Deus",
17.           new XElement("Nome", "Poseidon"),
18.           new XElement("Dominio", "Mar"),
19.           new XElement("Simbolo", "Tridente")),
20.         new XElement("Deus",
21.           new XElement("Nome", "Hades"),
22.           new XElement("Dominio", "Submundo"),
23.           new XElement("Simbolo", "Elmo da escuridão"))
24.       )
25.     );
26.
27.     // Consultando o documento XML utilizando LINQ to XML
28.     var deusesDoMar = from deus in documentoMitologiaGrega.Descendants
("Deus")
29.                 where (string)deus.Element("Dominio") == "Mar"
30.                 select deus;
31.
32.     // Imprimindo os deuses do mar
33.     Console.WriteLine("Deuses do Mar:");
34.     foreach (var deus in deusesDoMar)
35.     {
36.       Console.WriteLine(deus.Element("Nome").Value);
37.     }
38.   }
39. }
```

Figura 155 – LINQ to XML: exemplo de utilização

No código-fonte apresentado, um XDocument é criado (linha 10) para representar a mitologia grega, especificamente alguns deuses e seus domínios correspondentes. O XDocument tem um elemento raiz chamado Deuses, que contém elementos filhos, cada um representando um deus específico com seus atributos, como "Nome", "Dominio" e "Simbolo". Criado o documento XML, uma consulta LINQ to XML recupera todos os deuses cujo domínio seja o mar.

Na linha 28 vemos que essa consulta se utiliza da sintaxe de consulta do LINQ, onde os elementos são filtrados pela tag Dominio com valor Mar. Por fim, os deuses do mar encontrados pela consulta são impressos no console, demonstrando assim a utilização prática do LINQ to XML para criar e consultar documentos XML em um contexto relacionado à mitologia grega.

Nas figuras 154 e 155 utilizamos consultas simples em LINQ, porém podemos aumentar sua complexidade e especificidade utilizando consultas compostas, que adotam diversos operadores.

```
1.        // Realizando uma consulta composta com LINQ to XML
2.        var deusesOrdenados = documentoMitologiaGrega.Descendants("Deus")
3.           .Where(deus => (string)deus.Element("Dominio") == "Mar" ||
(string)deus.Element("Dominio") == "Céu")
4.           .OrderBy(deus => (string)deus.Element("Nome"))
5.           .Select(deus => new
6.           {
7.              Nome = (string)deus.Element("Nome"),
8.              Simbolo = (string)deus.Element("Simbolo")
9.           })
10.          .ToList();
```

Figura 156 – LINQ to XML: usando consulta composta

Aqui fizemos uma consulta composta utilizando LINQ to XML, filtramos os deuses que têm como domínio o Mar ou o Céu (linha 3), ordenamos os deuses resultantes por nome em ordem ascendente (linha 4) e projetamos uma lista com objetos anônimos contendo os elementos Nome e Simbolo de cada deus (linha 5).

O quadro 34 descreve as principais operações e consultas do LINQ.

Quadro 34 – LINQ: operações e consultas

Operação	Descrição
Select	Projeção de dados. Permite selecionar uma ou mais propriedades de objetos em uma coleção
Where	Filtra a coleção com base em uma condição especificada
OrderBy / OrderByDescending	Ordena a coleção com base em uma chave especificada, em ordem crescente ou decrescente
GroupBy	Agrupa elementos da coleção pela chave especificada, criando grupos de elementos relacionados
Join	Combina elementos de duas coleções com base em uma chave comum, gerando um resultado com as combinações desejadas

Operação	Descrição
Count	Conta o número de elementos na coleção que atendem a uma condição específica
Sum / Average / Min / Max	Realiza operações de agregação na coleção, como soma, média, mínimo e máximo de valores numéricos
Any	Verifica se algum elemento da coleção satisfaz uma condição especificada
All	Verifica se todos os elementos da coleção satisfazem uma condição especificada
Distinct	Retorna elementos distintos da coleção, removendo duplicatas
Take / Skip	Retorna uma quantidade específica de elementos da coleção ou pula uma quantidade especificada de elementos
First / FirstOrDefault	Retorna o primeiro elemento da coleção que atende a uma condição especificada. Se nenhum elemento for encontrado, pode retornar um valor-padrão
Single / SingleOrDefault	Retorna um único elemento da coleção que atende a uma condição especificada. Se houver mais de um ou nenhum elemento que satisfaça a condição, pode lançar uma exceção
ToList / ToArray	Converte a coleção em uma lista ou array, permitindo maior flexibilidade para trabalhar com os dados

7.3 Disposal e Garbage Collection

Na plataforma .NET – da qual C# é uma das linguagens principais – a gestão de memória é tarefa essencial, que garante a eficiência e a estabilidade das aplicações; e essa gestão é facilitada por dois conceitos-chave: Garbage Collection (GC) e Disposal. O primeiro é um processo automático que identifica e libera memória não mais acessível ou necessária para o programa. Em linguagens sem GC, como C++, a responsabilidade de alocar e desalocar memória recai sobre o programador, o que pode levar a erros como vazamentos de memória ou acesso a uma memória já liberada.

No .NET o GC rastreia os objetos alocados dinamicamente e determina se eles ainda são alcançáveis. Se um objeto não for mais referenciado por outros objetos ou pelo código, é considerado coletável e sua memória pode ser liberada. O GC opera em ciclos e tem várias gerações para otimizar a coleta e minimizar o impacto no desempenho da aplicação.

Lembrete

Disposal é uma abordagem complementar ao GC e se refere à liberação explícita de recursos não gerenciados, como conexões de banco de dados, handles de arquivos (conforme o tópico 3.13), ou recursos de rede.

Esses recursos não são gerenciados pelo GC e, portanto, não são automaticamente liberados quando o objeto que os detém é coletado. A interface IDisposable em .NET fornece um mecanismo para classes gerenciarem a liberação de tais recursos. Ao implementar essa interface, as classes fornecem um método Dispose, que pode ser chamado para liberar explicitamente recursos não gerenciados.

O uso adequado do método Dispose é crucial para garantir que os recursos sejam liberados em tempo hábil e para evitar bloqueios de recursos ou vazamentos que possam afetar a estabilidade do sistema. Para facilitar seu uso, C# introduziu a instrução using, garantindo que o método Dispose seja chamado automaticamente ao final do escopo do objeto.

7.4 Debugging e tracing

No desenvolvimento de software é imperativo que desenvolvedores dominem ferramentas e técnicas eficazes para diagnosticar, inspecionar e corrigir problemas em suas aplicações. Nesse contexto, dois conceitos-chave emergem: **debugging** e **tracing**.

Debugging, ou depuração, é o processo pelo qual desenvolvedores identificam e corrigem anomalias ou erros no código-fonte. No ambiente .NET, o Visual Studio é frequentemente empregado como a ferramenta principal de debugging, oferecendo uma interface rica que permite aos desenvolvedores executar código passo a passo, inspecionar valores de variáveis, avaliar expressões e estabelecer pontos de interrupção, que são locais específicos no código onde a execução será interrompida para inspeção. A depuração é vital não apenas para corrigir erros evidentes, mas também para entender melhor o fluxo de execução e as interações dentro do código, facilitando a identificação de problemas sutis ou comportamentos inesperados.

Já o tracing, ou rastreamento, refere-se ao registro ou monitoramento da execução do programa sem interrompê-lo. É frequentemente utilizado para capturar informações sobre a operação de um aplicativo em tempo real ou em ambientes onde a depuração interativa não é viável, como em produção. Em C# e no ambiente .NET, o namespace System.Diagnostics fornece uma série de classes e utilitários que auxiliam no rastreamento, frequentemente realizado ao se gerar logs ou mensagens que indiquem a aplicação, pontos de entrada ou saída de métodos, ou situações anômalas. A habilidade de ajustar o nível de detalhe desses registros, desde informações básicas até detalhes verbosos, permite que os desenvolvedores tenham uma visão granular do comportamento da aplicação em diferentes contextos.

O código-fonte da figura 157 define uma lista de deuses e os classifica entre os que pertencem ao submundo e os que pertencem ao Olimpo.

```
1.  using System;
2.  using System.Diagnostics;
3.
4.  public class Programa
5.  {
6.    public static void Main()
7.    {
8.      string[] deuses = { "Zeus", "Hermes", "Afrodite", "Hades" };
9.      foreach (var deus in deuses)
10.     {
11.       AnalisarDeus(deus);
12.     }
13.   }
14.
15.   public static void AnalisarDeus(string nome)
16.   {
17.     Debug.Assert(!string.IsNullOrEmpty(nome), "O nome do deus não
pode estar vazio.");
18.
19.     if (nome == "Hades")
20.     {
21.       Trace.TraceInformation("Iniciando a análise de um deus do
submundo: " + nome);
22.       Console.WriteLine($"{nome} é um deus do submundo.");
23.     }
24.     else
25.     {
26.       Trace.TraceInformation("Iniciando a análise de um deus do
Olimpo: " + nome);
27.       Console.WriteLine($"{nome} é um deus do Olimpo.");
28.     }
29.
30.     Trace.TraceInformation("Análise de " + nome + " concluída.");
31.   }
32. }
```

Figura 157 – Namespace System.Diagnostics: exemplo de Debug e Trace

O método AnalisarDeus é onde a maior parte da ação acontece. Antes de mais nada, há uma declaração Debug.Assert, uma ferramenta de debugging. Essa é a instrução para a execução (em modo de depuração) caso o nome do deus fornecido esteja vazio ou seja nulo, permitindo ao desenvolvedor identificar problemas antes que o código continue. É uma forma de garantir que uma condição esperada seja verdadeira e sinalizar imediatamente se não for.

Em seguida, dependendo do nome do deus fornecido, diferentes mensagens são registradas usando Trace.TraceInformation. Essas mensagens, que são exemplos de tracing, permitem aos desenvolvedores rastrear o fluxo de execução do programa. Por exemplo, quando o programa começa a analisar um deus, uma mensagem é registrada; e outra mensagem é registrada depois que a análise é concluída. Se o programa estiver configurado para registrar mensagens de rastreamento, elas aparecerão no output, fornecendo um registro contínuo das operações realizadas.

O uso combinado de Debug e Trace no código fornece uma ilustração clara de como o debugging e o tracing podem ser utilizados em conjunto para auxiliar no desenvolvimento e monitoramento de uma aplicação C#.

Observação

Enquanto o Debug enfrenta condições inesperadas durante o desenvolvimento, o Trace fornece uma forma contínua de monitorar a operação do programa em diferentes contextos. Ambas as práticas são complementares no desenvolvimento em C#.

Enquanto a depuração permite uma inspeção profunda e interativa do código durante o desenvolvimento e o teste, o rastreamento fornece insights contínuos sobre o comportamento do sistema em ambientes reais, ajudando a identificar problemas que podem não ser evidentes em ambientes de desenvolvimento. Em última análise, dominar as técnicas e ferramentas de debugging e tracing é essencial para qualquer desenvolvedor C#, pois essas práticas garantem não apenas a correção de erros, mas também a entrega de um software robusto, eficiente e, acima de tudo, confiável para os usuários finais.

7.5 Patterns

O conceito de **padrões de correspondência** (do inglês *pattern matching)*, embora presente em diversas linguagens de programação, ganhou relevância especial no C#, nas versões mais recentes do .NET Core e .NET 5, que integraram essa funcionalidade de maneira a permitir aos desenvolvedores identificar e extrair informações de dados mais facilmente do que usando abordagens tradicionais, como as instruções if e switch.

Padrões de correspondência podem ser compreendidos como um meio de testar a forma ou o padrão de um dado em vez de simplesmente seu valor, oferecendo uma maneira mais expressiva de trabalhar com dados e possibilitando verificar não só tipos, mas também valores e estruturas internas. Na prática, significa que os desenvolvedores podem realizar ações específicas com base em características detalhadas dos dados em vez de apenas seu tipo ou valor superficial.

Uma de suas vantagens no C# é a capacidade de simplificar o código. Antes da sua introdução, os desenvolvedores muitas vezes precisavam escrever várias instruções condicionais aninhadas para avaliar diferentes propriedades e valores de um objeto. Com os padrões de correspondência, esse processo é mais direto e menos propenso a erros, já que permite uma descrição mais clara e concisa das condições avaliadas.

Outra característica valiosa é a integração deles com outras funcionalidades, como as expressões de switch e as tuplas. Isso significa que não apenas é possível usá-los para identificar padrões específicos, mas também para destrinchar dados em componentes mais gerenciáveis, que podem ser processados ou transformados de acordo com as necessidades do desenvolvedor.

Para uma melhor compreensão, os padrões de correspondência podem ser divididos, conforme o quadro 35:

Quadro 35 – Padrões de correspondência: subdivisões

Padrão	Descrição
Tipo e Var Patterns	Permitem que os programadores verifiquem se um objeto é de determinado tipo; se for, o objeto pode ser automaticamente convertido para esse tipo, eliminando a necessidade de conversões explícitas
Constant Pattern	Permite verificar se uma variável corresponde a um valor constante específico
Recursive Patterns	Funcionalidade avançada que possibilita verificar propriedades e campos internos de um objeto. Por exemplo, pode verificar se um objeto contém certas propriedades com valores específicos
Property Pattern	Semelhante ao Recursive Pattern, foca a verificação das propriedades de um objeto
Tuple Pattern	Ideal para trabalhar com tuplas, permite que os desenvolvedores verifiquem múltiplos valores simultaneamente
Positional Pattern	Forma frequentemente usada com tipos que têm uma ordem ou posição específica para seus membros, como registros ou tuplas

Além disso, promovem uma abordagem mais segura de lidar com dados. Ao testar a forma ou o padrão de um dado, os desenvolvedores podem garantir que somente estão trabalhando com dados que atendem a critérios específicos, minimizando a chance de erro em tempo de execução.

Lembrete

Como vimos, padrões de correspondência são uma técnica de comparação que verifica se um valor corresponde a determinado padrão, sendo amplamente utilizado para simplificar o código e torná-lo mais legível, como em estruturas de controle de fluxo, possibilitando operações mais concisas e expressivas em tipos de dados. Não os confunda com o que observamos no tópico 6.3 (design patterns), soluções generalizadas e reutilizáveis para problemas comuns encontrados no design de software que ajudam desenvolvedores a escrever códigos mais organizados, modulares e reutilizáveis, facilitando a manutenção e a extensibilidade do sistema.

```
1.  using System;
2.  namespace MitologiaGrega
3.  {
4.    public class Deus
5.    {
6.      public string Nome { get; set; }
7.      public string Dominio { get; set; }
8.      public bool Olimpiano { get; set; }
9.    }
10.   class Program
11.   {
12.     static void Main(string[] args)
13.     {
14.       Deus hades = new Deus { Nome = "Hades", Dominio = "Submundo",
Olimpiano = false };
15.       // 1. Tipo e Var Patterns
16.       if (hades is Deus deusVar)
17.       {
18.         Console.WriteLine($"{deusVar.Nome} é um Deus.");
19.       }
20.       // 2. Constante Pattern
21.       if (hades.Nome is "Hades")
22.       {
23.         Console.WriteLine($"Olá, {hades.Nome}!");
24.       }
25.       // 3. Recursive Patterns
26.       if (hades is { Olimpiano: false })
27.       {
28.         Console.WriteLine($"{hades.Nome} não é um Olimpiano.");
29.       }
30.       // 4. Property Pattern
31.       if (hades is { Dominio: "Submundo" })
32.       {
33.         Console.WriteLine($"{hades.Nome} é o Deus do {hades.Dominio}.");
34.       }
35.       // 5. Tuple Pattern
36.       var poderesZeus = ("Relâmpago", "Céu");
37.       switch (poderesZeus)
38.       {
39.         case ("Relâmpago", "Céu"):
40.           Console.WriteLine("Zeus tem o poder do relâmpago e governa o
céu.");
41.           break;
42.       }
43.
44.       // 6. Positional Pattern
45.       var athena = ("Atena", "Sabedoria");
46.       VerificarDeus(athena);
47.       static void VerificarDeus((string nome, string atributo) deus)
48.       {
49.         if (deus is ("Atena", "Sabedoria"))
50.         {
51.           Console.WriteLine($"{deus.nome} é a Deusa da {deus.atributo}.");
52.         }
53.       }
54.     }
55.   }
56. }
```

Figura 158 – Padrões de correspondência: exemplo de utilização

Iniciamos o código com a representação de um deus grego através da classe Deus, que tem atributos como nome, domínio e um indicativo de ser Olimpiano ou não. Usando o exemplo de Hades, a divindade do Submundo, primeiro verificamos se ele é do tipo Deus através do uso do padrão **is** Deus. Ao se confirmar que sim, ele é automaticamente convertido e atribuído à variável deusVar.

Em seguida, aproveitando o poder dos padrões de correspondência, indagamos diretamente se o nome de Hades corresponde à string Hades, uma simples porém eficiente forma de confirmar sua identidade. Avançamos no entendimento do personagem ao investigar se Hades é Olimpiano. Através do padrão recursivo, inspecionamos diretamente a propriedade Olimpiano e determinamos que ele não pertence ao conjunto de deuses do Olimpo. Já com o padrão de propriedade, mergulhamos mais profundamente em seu domínio, garantindo sua autoridade sobre o Submundo.

Voltando a Zeus, rei dos deuses, capturamos a essência de seus poderes e domínio em uma tupla. Com um **switch**, o padrão de correspondência permite afirmar de forma expressiva que Zeus tem o poder do relâmpago e governa o céu (linha 37). Por fim, para a divindade Atena, simbolizamos seu nome e atributo em uma tupla. Com a ajuda de uma função estática, verificamos e reconhecemos Atena como deusa da sabedoria, uma demonstração do poder do padrão posicional.

Padrões de correspondência são uma adição significativa ao conjunto de ferramentas disponíveis para desenvolvedores em C#, pois oferecem uma abordagem mais expressiva, concisa e segura para lidar com dados, permitindo que os programadores escrevam código mais eficiente e menos propenso a erros. À medida que o C# evolui, é provável que padrões de correspondência se tornem ainda mais difundidos, dada sua utilidade e eficácia em simplificar e melhorar a escrita de código.

7.6 Threading

Refere-se ao conceito de execução simultânea de múltiplas tarefas dentro de uma aplicação, onde cada tarefa é tratada como uma thread (ou linha de execução). Em C#, o suporte a esse conceito é majoritariamente fornecido pelo namespace System.Threading. No contexto de um programa, thread é a **menor unidade de um processo que pode operar de forma independente**. Um único processo pode conter várias threads, o que permite a **execução paralela** de tarefas, especialmente em sistemas com múltiplos núcleos de CPU.

A classe Thread é uma das principais ferramentas que permitem criar e gerenciar threads; por outro lado, para evitar o custo de criá-las e destruí-las manualmente, C# oferece o ThreadPool, um conjunto de threads pré-inicializadas prontas para executar tarefas.

Observação

Além da simples criação de threads, o .NET Framework 4 introduziu uma abordagem mais moderna e elevada para operações assíncronas e multithreaded, denominada Tasks, disponível através do namespace System.Threading.Tasks. Essa abordagem (detalhada no tópico 7.7) oferece uma maneira mais simplificada e eficaz de lidar com operações em paralelo e assíncronas.

Contudo, com a capacidade de executar múltiplas tarefas em paralelo, surgem desafios relacionados à sincronização e ao acesso concorrente a recursos. A seguir, alguns exemplos do dia a dia que revelam esses desafios:

- quando vários clientes tentam retirar dinheiro da mesma conta ao mesmo tempo, o sistema deve garantir que o saldo da conta seja atualizado corretamente para evitar que mais dinheiro seja retirado do que o disponível;

- quando um sistema tenta realizar transferências entre duas contas em ordens diferentes (conta A para conta B e vice-versa), e cada operação exige bloqueios exclusivos nas contas envolvidas, pode haver um impasse se as operações forem simultâneas e cada uma esperar a outra liberar o bloqueio;

- quando vários clientes tentam reservar o último assento disponível em um voo, o sistema precisa garantir que apenas um cliente consiga a reserva, enquanto os outros recebem uma mensagem informando que o assento não está mais disponível;

- durante promoções ou lançamentos de produtos populares, muitos usuários podem tentar comprar o mesmo item simultaneamente. O sistema deve gerenciar o estoque corretamente, garantindo que os pedidos sejam processados com base na disponibilidade e evitando vender mais itens do que o disponível em estoque;

- em ambientes de trabalho colaborativo, como Google Docs ou Microsoft Office Online, vários usuários podem editar um documento simultaneamente. O sistema deve garantir que as edições de um usuário não sobrescrevam indevidamente as de outro, mantendo o documento consistente para todos;

- em jogos de mundo aberto, onde vários jogadores podem interagir com os mesmos objetos ou NPCs (personagens não jogáveis), o servidor do jogo precisa sincronizar as ações para garantir que todos os jogadores vejam o mesmo estado do mundo em tempo real;

- em ambientes como hospitais ou departamentos de atendimento ao cliente, onde pessoas são atendidas com base em sua ordem de chegada ou prioridade, o sistema deve garantir que cada pessoa ou requisição seja atendida de forma justa e ordenada, mesmo que várias requisições sejam feitas quase simultaneamente.

Em todos os exemplos, é crucial que os sistemas sejam projetados para lidar com acessos concorrentes de maneira apropriada, garantindo a integridade dos dados, a justiça e uma experiência de usuário eficiente. Falhar em administrar adequadamente esses desafios pode resultar em erros, perda de dados ou insatisfação do cliente.

```
1.  using System;
2.  using System.Threading;
3.  public class Deuses
4.  {
5.     // Objetos de bloqueio para garantir que as mensagens sejam
enviadas e recebidas de forma síncrona
6.     private static readonly object _lockZeus = new object();
7.     private static readonly object lockHera = new object();
8.     public void ZeusEnviaMensagem()
9.     {
10.       // Bloqueio exclusivo para Zeus, garantindo que apenas ele possa
enviar sua mensagem de cada vez
11.       lock ( lockZeus)
12.       {
13.          Console.WriteLine("Zeus está preparando uma mensagem...");
14.          // Simulando um atraso para representar o tempo que Zeus leva
para preparar a mensagem
15.          Thread.Sleep(2000);
16.
17.          Console.WriteLine("Zeus enviou uma mensagem para Hera!");
18.       }
19.    }
20.    public void HeraResponde()
21.    {
22.       // Bloqueio exclusivo para Hera, garantindo que apenas ela possa
responder de cada vez
23.       lock (_lockHera)
24.       {
25.          Console.WriteLine("Hera está lendo a mensagem de Zeus...");
26.          // Simulando um atraso para representar o tempo que Hera leva
para ler a mensagem
27.          Thread.Sleep(1000);
28.
29.          Console.WriteLine("Hera respondeu a mensagem de Zeus!");
30.       }
31.    }
32.    public static void Main()
33.    {
34.       Deuses comunicacaoEntreDeuses = new Deuses();
35.       // Criando threads separadas para Zeus e Hera para simular suas
ações simultaneamente
36.       Thread threadZeus = new Thread(new ThreadStart(comunicacaoEntreDe
uses.ZeusEnviaMensagem));
37.       Thread threadHera = new Thread(new ThreadStart(comunicacaoEntreDe
uses.HeraResponde));
38.       // Iniciando as threads
39.       threadZeus.Start();
40.       threadHera.Start();
41.       // Aguardando as threads concluírem suas tarefas antes de
finalizar o programa
42.       threadZeus.Join();
43.       threadHera.Join();
44.    }
45. }
```

Figura 159 – Threading: exemplo de utilização

No código-fonte apresentado, a comunicação entre Zeus e Hera é simulada por ações em threads separadas. Usamos objetos de bloqueio, especificamente _lockZeus e _lockHera, para garantir que apenas um deus possa enviar ou responder a uma mensagem de cada vez, evitando possíveis condições de corrida. A função Thread.Sleep simula o tempo que Zeus e Hera gastariam preparando ou lendo uma mensagem.

Finalmente, no método Main iniciamos ambas as threads e as aguardamos terminar usando Join. A comunicação entre os deuses é então sequencialmente ordenada, primeiramente com Zeus enviando uma mensagem e, em seguida, Hera respondendo. Dada a ordem de execução das threads e o tempo de espera (Thread.Sleep) definido, a saída esperada no console seria como na figura 160.

```
Zeus está preparando uma mensagem...
Hera está lendo a mensagem de Zeus...
Zeus enviou uma mensagem para Hera!
Hera respondeu a mensagem de Zeus!
```

Figura 160 – Threading: resultado da execução do código-exemplo

Em sistemas multithreaded, a execução exata pode variar dependendo do agendamento do sistema operacional e de outros fatores. No entanto, devido aos bloqueios (lock) implementados no código, a ordem das mensagens dentro de cada função (Zeus e Hera) é garantida. Ou seja, "Zeus está preparando uma mensagem..." sempre virá antes de "Zeus enviou uma mensagem para Hera!", e "Hera está lendo a mensagem de Zeus..." sempre virá antes de "Hera respondeu a mensagem de Zeus!". Contudo, mensagens entre as duas funções podem se entrelaçar, dependendo de como as threads são agendadas para execução.

Ferramentas como Mutex (Mutual Exclusion) garantem que apenas uma thread tenha acesso a recursos específicos ou seções de código de cada vez. É um mecanismo de sincronização que garante a apenas uma thread acessar recursos ou seções de código específicos de cada vez. Da mesma forma, os conceitos de Semaphore, Monitor e lock limitam ou sincronizam o acesso de threads a determinadas partes do código ou recursos. O primeiro permite limitar o número de threads que podem acessar um recurso ou seção de código simultaneamente; os outros dois garantem que apenas uma thread execute um bloco de código específico de cada vez. Além disso, existem mecanismos como ManualResetEvent e AutoResetEvent que permitem a sinalização entre threads, garantindo que uma ou mais threads esperem até que recebam um sinal para continuar a execução.

No código-fonte da figura 161, o prestigioso Oráculo de Delfos é o recurso que todos os deuses desejam consultar. No entanto, existe uma limitação inerente: apenas um deus pode consultar o oráculo por vez, e há uma capacidade máxima na fila de espera. Assim, adotamos dois mecanismos de sincronização – Mutex e Semaphore – para modelar esse cenário. O Mutex (representado por mutexOraculo) garante que apenas um deus possa consultar o oráculo por vez, agindo como uma porta pela qual apenas um deus pode passar.

```
1.  using System;
2.  using System.Threading;
3.  public class OraculoDeDelfos
4.  {
5.     private static Mutex mutexOraculo = new Mutex();
6.     private static Semaphore semaforoFila = new Semaphore(2, 2); //
Somente 2 deuses podem esperar na fila ao mesmo tempo
7.     public void ConsultarOraculo(string nomeDeus)
8.     {
9.        Console.WriteLine($"{nomeDeus} deseja consultar o oráculo...");
10.       semaforoFila.WaitOne(); // O deus entra na fila se houver espaço
11.       Console.WriteLine($"{nomeDeus} está na fila para o oráculo...");
12.
13.       mutexOraculo.WaitOne(); // O deus aguarda sua vez para consultar
o oráculo
14.       Console.WriteLine($"{nomeDeus} está consultando o oráculo...");
15.       Thread.Sleep(2000); // Representa o tempo de consulta ao oráculo
16.
17.       Console.WriteLine($"{nomeDeus} terminou a consulta.");
18.
19.       mutexOraculo.ReleaseMutex(); // O deus libera o oráculo para o
próximo
20.       semaforoFila.Release(); // O deus sai da fila
21.    }
22.
```

Figura 161 – Threading: exemplo de uso de Mutex e Semaphore

Uma vez que um deus começa a consultar o oráculo, ele "trava" esse mutex, e qualquer outro deus que tente consultá-lo simultaneamente terá que esperar. Já o Semaphore (representado por semaforoFila) limita o número de deuses que podem ficar na fila de espera. No nosso cenário, somente dois deuses podem esperar ao mesmo tempo; se mais de dois tentarem, serão impedidos até que se abra espaço na fila.

```
1.     public static void Main()
2.     {
3.       OraculoDeDelfos oraculo = new OraculoDeDelfos();
4.
5.       Thread[] deuses = new Thread[4];
6.       deuses[0] = new Thread(() => oraculo.ConsultarOraculo("Zeus"));
7.       deuses[1] = new Thread(() => oraculo.ConsultarOraculo("Hera"));
8.       deuses[2] = new Thread(() => oraculo.ConsultarOraculo("Apolo"));
9.       deuses[3] = new Thread(() => oraculo.ConsultarOraculo("Atena"));
10.
11.      foreach (var deus in deuses)
12.      {
13.        deus.Start();
14.      }
15.
16.      foreach (var deus in deuses)
17.      {
18.        deus.Join();
19.      }
20.    }
21.  }
```

Figura 162 – Threading: método Main para Mutex e Semaphore

No método Main (figura 162) criamos threads representando deuses (Zeus, Hera, Apolo e Atena) que desejam consultar o oráculo. À medida que o programa é executado, é possível vê-los entrando na fila, aguardando sua vez, consultando o oráculo e depois saindo – tudo isso respeitando as restrições do Mutex e do Semaphore. Esse código exemplifica como os mecanismos de sincronização em programação multithread podem garantir acesso ordenado e limitado a um recurso, representado pelo Oráculo de Delfos, no contexto da mitologia grega.

Dado o funcionamento de Mutex e Semaphore no código, bem como a ordem em que se iniciam as threads, a saída no console pode variar ligeiramente devido ao agendamento de threads do sistema operacional. No entanto, há uma saída possível (e provável), apresentada pela figura 163.

```
Zeus deseja consultar o oráculo...
Hera deseja consultar o oráculo...
Apolo deseja consultar o oráculo...
Atena deseja consultar o oráculo...
Zeus está na fila para o oráculo...
Hera está na fila para o oráculo...
Zeus está consultando o oráculo...
Zeus terminou a consulta.
Hera está consultando o oráculo...
Apolo está na fila para o oráculo...
Hera terminou a consulta.
Apolo está consultando o oráculo...
Apolo terminou a consulta.
Atena está consultando o oráculo...
Atena terminou a consulta.
```

Figura 163 – Threading: resultado da execução do novo código-exemplo

A seguir, o que podemos inferir da saída da figura 163:

1. Todos os deuses expressam desejo em consultar o oráculo quase imediatamente.

2. Zeus e Hera conseguem entrar na fila imediatamente, pois o semáforo permite que dois deuses esperem na fila ao mesmo tempo.

3. Zeus começa a consulta enquanto Hera espera.

4. Depois que Zeus termina, Hera começa a consulta.

5. Enquanto Hera consulta o oráculo, Apolo consegue entrar na fila.

6. O mesmo processo se repete para Apolo e depois Atena.

No entanto, devido à natureza concorrente das threads e ao agendamento não determinístico do sistema operacional, a ordem exata pode variar em diferentes execuções; por exemplo, Hera pode começar a consulta antes de Zeus em uma execução distinta. Porém as regras estabelecidas pelo Mutex e Semaphore sempre serão seguidas: apenas um deus consulta o oráculo por vez e no máximo dois deuses podem estar na fila simultaneamente.

Saiba mais

Um guia prático para se aprofundar nos desafios da programação assíncrona, paralela e multithreaded em C# é o livro a seguir, que tem tópicos avançados nessa seara, expostos de forma clara e didática:

CLEARY, S. *Concurrency in C# cookbook*: asynchronous, parallel, and multithreaded programming. Sebastopol: O'Reilly Media, 2019.

Usar threading de forma adequada pode melhorar significativamente o desempenho de um programa, permitindo que tarefas demoradas sejam concluídas em paralelo com outras operações. No entanto, o threading também pode aumentar a complexidade e gerar potenciais problemas, como condições de corrida, deadlocks e outras questões de sincronização.

7.7 Tasks

Task é uma unidade de trabalho que pode ser executada de forma assíncrona. A biblioteca de tarefas paralelas, conhecida como Task Parallel Library (TPL), introduziu o conceito de tarefa como peça fundamental para a programação assíncrona e paralela.

No desenvolvimento tradicional de software, as operações são executadas sequencialmente. Agora imagine uma aplicação que solicite dados de um servidor remoto. Se essa solicitação fosse executada de forma síncrona, a aplicação ficaria esperando a resposta do servidor antes de prosseguir com outras tarefas, potencialmente fazendo o usuário esperar e dando a sensação de um aplicativo travado. A programação assíncrona, por outro lado, permite que a aplicação inicie uma operação – como a solicitação ao servidor – e continue executando outras tarefas enquanto espera pela conclusão. Quando a operação assíncrona é concluída, a aplicação é notificada e pode processar os resultados, tudo isso sem ter bloqueado a execução principal da aplicação.

A programação paralela se relaciona à execução simultânea de múltiplas operações. Em sistemas modernos, com múltiplos núcleos de processador, é possível executar várias tarefas ao mesmo tempo, literalmente em paralelo. Por exemplo, se tivéssemos um grande conjunto de dados a processar, poderíamos dividi-lo em partes menores e processar várias partes simultaneamente, em diferentes núcleos. A programação paralela permite que as aplicações tirem proveito dessa capacidade, acelerando operações que seriam demoradas se executadas sequencialmente.

A TLP fornece ferramentas e estruturas para facilitar tanto a programação assíncrona quanto a paralela; com ela os desenvolvedores podem iniciar tarefas assíncronas, aguardar sua conclusão e paralelizar operações, como loops, para que sejam executadas simultaneamente em múltiplos núcleos.

Tanto threads (detalhadas no tópico 7.6) quanto tasks lidam com a execução concorrente de código, mas abordam essa concorrência de maneiras diferentes e servem a propósitos distintos. A classe Thread representa uma execução singular e independente no nível do sistema operacional. Ao criar e iniciar uma nova instância de Thread, estamos literalmente solicitando que o sistema operacional aloque recursos para uma nova thread de execução. Isso dá ao desenvolvedor um **controle granular sobre a execução**, mas com esse controle vem a responsabilidade de gerenciar explicitamente todos os aspectos dela, como sincronização e possíveis conflitos dessa thread.

Task é uma abstração de alto nível sobre a execução assíncrona e muitas vezes pode ser vista como representação de uma operação ocorrendo em segundo plano. Entretanto, uma Task não é uma thread em si; em vez disso, a TPL utiliza um pool de threads para executar tasks. Pool de threads é uma coleção pré-inicializada de threads prontas para executar tarefas, sem a necessidade de criar novas threads do zero a cada vez que uma operação concorrente é necessária. Assim, ao criar e iniciar uma Task, ela será agendada para execução em uma das threads desse pool, sem a necessidade de criar explicitamente uma nova thread do sistema operacional.

Isso traz várias vantagens. Primeiramente, reutilizar threads de um pool é **geralmente mais eficiente** do que criar e destruir constantemente novas threads. Além disso, a abstração fornecida pela Task permite uma **programação assíncrona mais fácil e intuitiva**. Por outro lado, a TPL se encarrega de muitos detalhes de gerenciamento e sincronização, permitindo que os desenvolvedores se concentrem na lógica do aplicativo.

O tipo Task é a representação principal de uma unidade de trabalho que pode estar em andamento ou já concluída. Quando essa unidade de trabalho é projetada para retornar um valor após sua conclusão, utilizamos **Task<T>**, onde T especifica o tipo do valor a ser retornado. Nesse paradigma, C# oferece dois modificadores-chave: **async** e **await**. O async permite definir métodos que retornarão tasks; e dentro desses métodos a palavra-chave await chama outros métodos que retornam tasks, pausando temporariamente a execução do método atual até que a task invocada seja concluída, mas sem bloquear a thread em que está sendo executada.

Iniciar uma task é uma operação simples, geralmente realizada usando o método Task.Run(), que pega um delegado e o executa em uma thread separada, oriunda do pool de threads. Quando uma task é finalizada, é importante considerar o tratamento adequado, especialmente porque elas podem lançar exceções que, se lançadas dentro de uma tarefa, serão retransmitidas ao tentarmos acessar o resultado da task ou explicitamente esperar por sua conclusão.

Um aspecto crucial das tarefas em C# é a capacidade de cancelá-las. O cancelamento é implementado pela classe CancellationTokenSource, que fornece um token de cancelamento. O código executado dentro da task pode então verificar periodicamente esse token para verificar se um cancelamento foi pedido, permitindo um encerramento simples e objetivo.

Além disso, o conceito de continuação é fundamental ao trabalhar com tarefas. Concluída uma task, pode-se desejar que outro bloco de código seja executado imediatamente depois. Isso é gerenciado usando o método ContinueWith, que permite encadear tarefas, garantindo que certo código seja executado após a conclusão de uma tarefa específica.

A figura 164 ilustra tarefas no C#.

```
1.  using System;
2.  using System.Threading.Tasks;
3.
4.  class Program
5.  {
6.     static async Task Main(string[] args)
7.     {
8.        // Iniciamos uma tarefa para buscar informações sobre Zeus.
9.        Task<string> tarefaZeus = ObterInformacoesDeusAsync("Zeus");
10.
11.       // Enquanto isso, continuamos executando outras operações, como
buscar informações sobre Hera.
12.       string infoHera = await ObterInformacoesDeusAsync("Hera");
13.
14.       Console.WriteLine(infoHera);
15.
16.       // Agora, esperamos que a tarefa referente a Zeus termine.
17.       string infoZeus = await tarefaZeus;
18.
19.       Console.WriteLine(infoZeus);
20.    }
21.
22.    static async Task<string> ObterInformacoesDeusAsync(string
nomeDeus)
23.    {
24.       // Simulando um atraso para representar uma operação demorada,
como uma busca em banco de dados ou requisição à uma API.
25.       await Task.Delay(2000);
26.
27.       // Retornando algumas informações fictícias sobre os deuses
gregos.
28.       if (nomeDeus == "Zeus")
29.       {
30.          return "Zeus é o deus do céu e governante do Olimpo.";
31.       }
32.       else if (nomeDeus == "Hera")
33.       {
34.          return "Hera é a rainha dos deuses e esposa de Zeus.";
35.       }
36.       else
37.       {
38.          return $"Informações sobre {nomeDeus} não encontradas.";
39.       }
40.    }
41. }
```

Figura 164 – Task: exemplo de utilização

Na linha 6 definimos o método Main como uma task. Não é obrigatório fazer isso para usar tarefas, exceto se o programador quiser usar o await diretamente dentro do método Main, que é o caso. Antes das versões mais recentes do C#, o método Main não poderia ser assíncrono. Se quiséssemos chamar

um método assíncrono a partir do Main, precisaríamos fazê-lo de forma indireta e, possivelmente, usar métodos como Task.Wait ou Task.Result para esperar pela conclusão da tarefa, o que não é recomendado, pois pode causar bloqueios.

O C# 7.1 introduziu o suporte para um método Main assíncrono, permitindo que os desenvolvedores definissem o Main como async Task ou async Task<int>, facilitando a chamada de métodos assíncronos diretamente do Main usando o await, sem a necessidade de bloquear a thread principal. Na linha 22 declaramos uma função assíncrona chamada ObterInformacoesDeusAsync, que simula uma operação demorada usando Task.Delay(2000);. Isso poderia, por exemplo, representar uma busca em um banco de dados ou uma requisição a uma API externa. Dependendo do nome do deus passado para essa função, ela retorna uma informação fictícia sobre ele.

Na função Main iniciamos a tarefaZeus para buscar informações sobre Zeus – operação iniciada e processada em segundo plano. Não esperamos que ela termine imediatamente; em vez disso, seguimos em frente e solicitamos informações sobre Hera, esperando pela resposta usando a palavra-chave **await**. Ao usá-la, o programa pausa a função Main (sem bloquear a thread) até que a tarefa para Hera esteja completa; uma vez completa, imprimimos as informações sobre ela ("Hera é a rainha dos deuses e esposa de Zeus."). Na sequência voltamos à tarefaZeus e esperamos que ela termine (se ainda não tiver terminado) usando novamente await. Terminada a tarefa, as informações sobre Zeus são impressas ("Zeus é o deus do céu e governante do Olimpo.").

7.8 Funções e padrões assíncronos

O desenvolvimento de software frequentemente exige que operações de longa duração sejam executadas sem interromper o fluxo principal da aplicação, especialmente quando se trata de interfaces de usuário ou serviços que precisam responder rapidamente. Em C#, antes da popularização das palavras-chave async e await, duas abordagens predominavam no que diz respeito à programação assíncrona: o Event-Based Asynchronous Pattern (EAP) e o Asynchronous Programming Model (APM).

O primeiro é uma abordagem orientada a eventos para operações assíncronas. Como o nome sugere, esse padrão **depende de eventos para notificar** o chamador sobre o início, progresso e conclusão de uma operação assíncrona. Os componentes que utilizam EAP geralmente adotam um método Start para iniciar a operação assíncrona e um evento Completed para indicar que ela terminou. Além disso, podem incluir eventos como ProgressChanged para fornecer atualizações intermediárias.

Um aspecto fundamental do EAP é oferecer uma maneira de cancelar operações em andamento através de métodos como Cancel. Ainda que o EAP torne certas operações mais intuitivas – principalmente para desenvolvedores familiarizados com a programação orientada a eventos –, pode ser desafiador gerenciar múltiplas operações assíncronas simultâneas. Já o APM é frequentemente reconhecido por sua abordagem de pares Begin e End; em vez de depender de eventos, utiliza callbacks para notificar o término de operações.

Abordagem baseada em callbacks refere-se a um padrão de projeto no qual uma função é passada como argumento (ou parâmetro) para outra função, para ser executada em um momento posterior. Esse retorno de chamada (ou callback) geralmente é invocado após a conclusão de uma operação assíncrona, permitindo que o código continue a execução enquanto aguarda o resultado de uma tarefa demorada – como leitura de arquivos, solicitações de rede, entre outras. A ideia é que uma operação assíncrona seja iniciada com um método que tenha o prefixo Begin e seja concluída com um método de prefixo End.

O método Begin normalmente aceita um delegate de callback e um objeto de estado, passado para o método End quando a operação é concluída. Embora o APM forneça um grande controle sobre as operações assíncronas, pode se tornar complexo rapidamente, especialmente ao encadear várias operações assíncronas.

Ambos os padrões, EAP e APM, têm seus méritos. O EAP, com sua orientação a eventos, pode ser mais intuitivo para certos cenários, especialmente quando as atualizações intermediárias são essenciais. Já o APM, com sua abordagem baseada em callbacks, oferece uma granularidade que pode ser benéfica em situações que exigem controle meticuloso. No entanto, com a evolução do C# e a introdução das palavras-chave async e await, a programação assíncrona tornou-se mais acessível e direta.

Apesar desses novos paradigmas, é essencial reconhecer e entender EAP e APM, pois eles ainda existem em muitos **sistemas legados** e fornecem uma base para apreciar as inovações mais recentes em programação assíncrona em C#.

Saiba mais

É importante conhecer programação para sistemas legados, dada a prevalência de sistemas mais antigos ainda vitais para muitas organizações. A manutenção e a atualização desses sistemas requerem um entendimento das práticas de programação e arquiteturas usadas no passado. A seguir, uma boa recomendação de leitura sobre legados:

FEATHERS, M. C. *Trabalho eficaz com código legado*. Porto Alegre: Bookman, 2013.

8 CLASSES E ALGORITMOS

Uma característica distintiva do C# é sua extensa biblioteca-padrão, pois oferece múltiplas estruturas que auxiliam desenvolvedores a manipular e organizar dados. Em relação às que tratam dados de maneira sequencial, o C# oferece recursos que facilitam a inserção, remoção e busca de informações, sendo fundamentais em ambientes de desenvolvimento onde a sequência e a estrutura dos dados desempenham um papel crucial na lógica do programa. Com elas os desenvolvedores podem construir sistemas que lidam com volumes significativos de dados, otimizando o tempo de desenvolvimento e a manutenção de programas.

No âmbito da recuperação rápida de dados, o C# é equipado com estruturas otimizadas, eficazes para associar dados e verificar sua presença. Essas estruturas permitem que os desenvolvedores realizem operações de forma ágil e precisa, independente do tamanho ou complexidade dos conjuntos de dados em questão. Em representações hierárquicas, o C# disponibiliza estruturas que possibilitam aos desenvolvedores capturar e manipular relações entre os dados, o que é útil se a relação entre os elementos for determinante para a funcionalidade do sistema.

Veremos que C# é uma linguagem de programação que fornece um conjunto completo de ferramentas e estruturas, abrangendo desde simples gestões sequenciais até representações hierárquicas mais elaboradas. Com essas capacidades, a linguagem permite que os desenvolvedores se concentrem na implementação de lógicas e soluções, enquanto a linguagem facilita a manipulação e organização dos dados.

8.1 Ordenação de vetores e listas – seleção, inserção, bubble sort, quick sort

Ordenar vetores e listas é uma atividade fundamental na programação e na ciência da computação, e diversos algoritmos foram desenvolvidos ao longo dos anos para isso. Em C#, assim como em muitas outras linguagens de programação, temos a liberdade de implementar esses algoritmos para ordenar estruturas como vetores (arrays) e listas.

O algoritmo de Seleção (selection sort) opera buscando, iterativamente, o menor (ou maior) elemento do vetor ou lista e colocando-o na posição correta (Cormen *et al.*, 2002). Em outras palavras, durante cada passagem pelo vetor, o algoritmo seleciona o menor item e faz uma troca com o item da primeira posição ainda não ordenada. Apesar de simples, Selection Sort não é o algoritmo mais eficiente para grandes volumes de dados; o Inserção (Insertion Sort) é outro algoritmo simples, mas que sob determinadas circunstâncias pode ser mais eficaz. A ideia aqui é, a cada iteração, pegar um elemento e inseri-lo na posição correta entre os elementos anteriormente analisados (Ziviani, 2004). Imagine que você está jogando cartas e, a cada nova carta recebida, você a insere na posição correta da sua mão, mantendo a sequência ordenada; essa é essencialmente a lógica por trás do Insertion Sort.

Bubble Sort é talvez um dos algoritmos de ordenação mais conhecidos, embora não seja dos mais eficientes. Ele trabalha repetidamente percorrendo a lista, comparando pares de elementos adjacentes e trocando-os se estiverem na ordem errada. O processo se repete até que nenhuma troca seja necessária, indicando que a lista está ordenada (Sedgewick, 2013). A denominação Bubble provém da forma como os valores maiores "borbulham" para as posições finais da lista.

Finalmente, Quick Sort é um algoritmo de ordenação mais avançado e eficiente para conjuntos maiores de dados (Cormen *et al.*, 2002), baseando-se em uma técnica de divisão e conquista. O processo envolve a escolha de um "pivô" e, em seguida, a divisão da lista em duas partes: uma com elementos menores que o pivô e outra com elementos maiores; e o algoritmo é então aplicado recursivamente a essas duas sublistas. Quando bem implementado, o Quick Sort pode ser significativamente mais rápido que os algoritmos mencionados, especialmente para listas grandes.

Esses algoritmos podem ser implementados utilizando estruturas de dados nativas, como arrays e List<T>, bem como operações básicas de iteração e comparação. Embora existam métodos integrados no framework .NET para ordenação, compreender a lógica e o funcionamento interno desses algoritmos é crucial para qualquer desenvolvedor, pois fornece insights sobre eficiência e otimização. Ao lidar com a ordenação no desenvolvimento do dia a dia, é importante considerar tanto a natureza dos dados quanto o volume para escolher o algoritmo mais adequado.

O quadro 36 pode guiar decisões, mas é importante lembrar que a eficiência dos algoritmos pode variar com base em implementações específicas, otimizações e características intrínsecas dos dados (por exemplo, se já estão parcialmente ordenados). Além disso, outros algoritmos de ordenação não mencionados aqui (como Merge Sort e Heap Sort) também podem ser relevantes dependendo do contexto.

Quadro 36 – Algoritmos de ordenação

Algoritmo	Natureza da operação	Volume recomendado de dados
Selection Sort	Adequado para operações simples, sem precisar de estabilidade na ordenação	Pequeno a médio. Não recomendado para grandes volumes devido à sua eficiência
Insertion Sort	Mais eficiente para listas parcialmente ordenadas. Mantém estabilidade na ordenação (elementos iguais mantêm ordem original)	Pequeno, em especial quando a lista já está parcialmente ordenada
Bubble Sort	Apesar de sua simplicidade, é menos eficiente. Mantém estabilidade na ordenação	Pequeno. Geralmente superado em eficiência por outros métodos, mesmo para pequenos volumes
Quick Sort	Muito eficiente para grandes volumes. Não garante estabilidade na ordenação. Pode ser otimizado dependendo da escolha do pivô	Médio a grande. É um dos algoritmos mais rápidos para grandes volumes de dados, mas sua eficiência pode variar dependendo da implementação e escolha do pivô

Vamos explorar ainda alguns exemplos reais de aplicação desses algoritmos de ordenação. Em sistemas simples de gerenciamento de bibliotecas, onde o acervo é limitado e a frequência de novas adições e pesquisas é baixa, o Selection Sort pode ordenar livros por título ou autor, dada a simplicidade do algoritmo e a ausência de necessidade de alto desempenho.

Agora imagine um jogo digital cujos envolvidos recebem cartas de forma sequencial e precisam mantê-las ordenadas na mão. Como elas são recebidas uma de cada vez, e a mão do jogador geralmente é pequena, o Insertion Sort é eficiente para inserir cada nova carta na posição correta. Em sistemas didáticos ou educativos – cujo objetivo é ensinar os fundamentos da ordenação e cuja visualização do processo é mais importante que a eficiência –, o Bubble Sort pode ser utilizado, dado que sua lógica é fácil de entender e visualizar.

Por fim, em um e-commerce com milhares de produtos, ao se fazer uma pesquisa, os resultados precisam ser ordenados por relevância, preço ou avaliação do cliente rapidamente. Dado o grande volume de produtos e a necessidade de respostas rápidas, o Quick Sort pode ser uma excelente escolha, pois é eficiente em grandes conjuntos de dados e geralmente supera os algoritmos de ordenação quadrática – lembrando sempre que, mesmo com esses exemplos, a decisão sobre qual algoritmo utilizar deve considerar o cenário específico, a natureza dos dados, a frequência de operações de ordenação e outros fatores relevantes para o sistema em questão.

Já o Bubble Sort (também conhecido como ordenação por bolha) é um dos algoritmos de ordenação mais simples, tanto em sua compreensão quanto implementação, que compara pares adjacentes de elementos em uma lista e troca-os de lugar se estiverem na ordem errada, em um processo que se repete por toda a lista diversas vezes, até que esteja completamente ordenada. É chamado Bubble (bolha em inglês) porque os elementos da lista "flutuam" gradualmente para suas posições corretas, assim como as bolhas sobem na água.

```
1.  using System;
2.
3.  namespace BubbleSortMitologia
4.  {
5.    public class Program
6.    {
7.      public static void Main(string[] args)
8.      {
9.        // Lista de deuses gregos a serem ordenados.
10.       string[] deuses = { "Zeus", "Hera", "Ares", "Apolo",
"Afrodite", "Hermes", "Atena" };
11.
12.       OrdenacaoPorBolha(deuses);
13.
14.       // Exibir a lista de deuses ordenados.
15.       foreach (string deus in deuses)
16.       {
17.         Console.WriteLine(deus);
18.       }
19.     }
20.
21.     public static void OrdenacaoPorBolha(string[] deuses)
22.     {
23.       int tamanho = deuses.Length;
24.
25.       for (int i = 0; i < tamanho - 1; i++)
26.       {
27.         for (int j = 0; j < tamanho - i - 1; j++)
28.         {
29.           if (string.Compare(deuses[j], deuses[j + 1]) > 0)
30.           {
31.             // Trocar deuses[j] com deuses[j + 1]
32.             string temp = deuses[j];
33.             deuses[j] = deuses[j + 1];
34.             deuses[j + 1] = temp;
35.           }
36.         }
37.       }
38.     }
39.   }
40. }
```

Figura 165 – BubbleSort: implementação

O código-fonte apresentado implementa o Bubble Sort em C# para ordenar uma lista de nomes de deuses da mitologia grega. O programa começa definindo essa lista (linha 10) e em seguida passa-a para a função OrdenacaoPorBolha, responsável pela ordenação (linha 12). A lógica da função OrdenacaoPorBolha é iterar a lista diversas vezes. Em cada iteração, o algoritmo compara pares adjacentes de deuses e troca-os de posição se estiverem fora de ordem. A cada passagem pela lista, o maior nome (em termos de ordem alfabética) é "bolha" até sua posição correta. Como resultado, após a primeira passagem o maior nome está na última posição; após a segunda, o segundo maior nome está na penúltima posição, e assim por diante.

Os laços **for** aninhados no código (linhas 25 e 27) iteram os elementos da lista e fazem as comparações necessárias. A função string.Compare (como vimos no tópico 4.4) compara dois nomes: se o resultado for positivo, significa que o primeiro nome vem depois do segundo em ordem alfabética e eles precisam ser trocados. Após todas as iterações, a lista de deuses está ordenada, e o programa principal (Main) exibe os deuses em ordem (linha 15). Precisamente, a lista original Zeus, Hera, Ares, Apolo, Afrodite, Hermes, Atena (linha 10) fica como Afrodite, Apolo, Ares, Atena, Hera, Hermes, Zeus após o término da execução.

A biblioteca-padrão (.NET Framework para versões anteriores ao .NET Core e .NET Core/.NET 5+ para versões mais recentes) oferece mecanismos para ordenar coleções, como List<T>.Sort() e Array.Sort(). No entanto, esses métodos utilizam algoritmos otimizados de ordenação em sua maioria baseados em uma variação do QuickSort (para tipos de valor) e uma combinação de QuickSort, HeapSort e InsertionSort (para tipos de referência). Para pequenos arrays (com menos de 16 elementos), o InsertionSort pode ser usado.

8.2 Pilhas e filas – Torre de Hanói

Pilhas e filas são estruturas de dados fundamentais, que armazenam e gerenciam informações em uma ordem específica. Ambas têm aplicações em diversos domínios, desde o processamento de dados até operações em sistemas e aplicações de software.

Uma pilha, como o nome sugere, pode ser vista como uma pilha de pratos. Imagine que você está empilhando. O prato que você coloca por último é o primeiro que tira quando começa a desempilhar – comportamento conhecido como *last in, first out* (Lifo), ou seja, o último item que entra é o primeiro a sair. Em termos técnicos, uma pilha permite duas operações principais: **push**, que adiciona um elemento no topo da pilha; e **pop**, que remove o elemento do topo.

Pilhas são comumente usadas em operações como a avaliação de expressões matemáticas e o rastreamento de chamadas de funções em linguagens de programação. Uma das aplicações mais intuitivas da pilha é o recurso "desfazer" (ou "undo"), presente em muitos softwares, desde editores de texto até softwares de design gráfico. Quando você executa uma série de ações – como digitar, apagar ou desenhar –, elas são empilhadas. Se você decide desfazer sua ação mais recente, a operação no topo da pilha é revertida. Quanto mais vezes você pressiona "desfazer", mais ações são desempilhadas e revertidas em ordem. Da mesma forma, muitos navegadores de internet utilizam pilhas para gerenciar o histórico de navegação. Quando você clica no botão "voltar", é para o site no topo da pilha que retorna.

Torre de Hanói é um clássico problema de quebra-cabeça que se baseia no uso de pilhas. O jogo consiste em três hastes (ou pinos) e um número de discos de tamanhos diferentes, que podem deslizar para qualquer haste. O quebra-cabeça começa com os discos em uma pilha de ordem crescente em uma haste (com o menor no topo), formando uma espécie de pirâmide.

O objetivo do quebra-cabeça é mover a pilha inteira para outra haste, obedecendo às seguintes regras:

- Apenas um disco pode ser movido de cada vez.
- Cada movimento consiste em pegar o disco superior de uma das hastes e deslizá-lo para o topo de outra haste.
- Um disco não pode ser colocado em cima de um disco menor.

Do ponto de vista computacional, a forma mais elegante de resolver o quebra-cabeça é um algoritmo recursivo. **Recursão** é um conceito fundamental em ciência da computação e matemática, e refere-se ao processo em que uma função chama a si mesma em sua definição. Na programação, uma função recursiva resolve uma tarefa em partes, invocando a mesma função com argumentos diferentes, até que a solução seja alcançada. Essa abordagem é especialmente útil para problemas que podem ser divididos em subproblemas menores, de natureza similar ao problema original.

Imagine um conjunto de caixas aninhadas, e cada uma contém uma caixa menor, exceto a menor delas. O ato de abrir cada caixa para encontrá-la e abrir a caixa subsequente é análogo à recursão, onde cada etapa do processo é uma repetição da etapa anterior, mas em escala reduzida. Uma característica importante de uma função recursiva é a presença de uma condição de encerramento, também conhecida como caso-base. Sem essa condição a função continuaria a chamar a si mesma indefinidamente, levando a um ciclo infinito.

Caso-base é o ponto em que a função retorna um valor sem fazer outra chamada recursiva. Por exemplo, ao calcular o fatorial de um número usando recursão, o caso-base é geralmente definido como o fatorial de 1 sendo igual a 1. Sem esse caso-base a função continuaria a tentar calcular o fatorial de números progressivamente menores, sem nunca chegar a uma conclusão.

A figura 166 ilustra um exemplo do quebra-cabeça com recursão. A solução para a Torre de Hanói é frequentemente implementada com uma abordagem recursiva, e durante esse processo a estrutura de pilha é intrínseca ao problema. Cada haste pode ser vista como pilha, onde os discos são empurrados e desempilhados conforme as regras do jogo. Ao mover os discos entre as hastes, estamos essencialmente realizando operações de push e pop em pilhas.

```
1.  using System;
2.  namespace TorreDeHanoi
3.  {
4.     class Program
5.     {
6.        // Método recursivo que resolve o quebra-cabeça da Torre de Hanói
7.        static void MoverDiscos(int quantidade, char origem, char
destino, char auxiliar)
8.        {
9.           // Se tivermos apenas 1 disco, movemos ele diretamente do pino
origem para o destino
10.          if (quantidade == 1)
11.          {
12.             Console.WriteLine($"Mova o disco 1 de {origem} para
{destino}");
13.             return;
14.          }
15.          // Movemos 'quantidade - 1' discos da origem para o pino
auxiliar, usando o destino como auxiliar
16.          MoverDiscos(quantidade - 1, origem, auxiliar, destino);
17.          // Agora, movemos o disco remanescente da origem para o destino
18.          Console.WriteLine($"Mova o disco {quantidade} de {origem} para
{destino}");
19.          // Finalmente, movemos os 'quantidade - 1' discos do pino
auxiliar para o destino
20.          MoverDiscos(quantidade - 1, auxiliar, destino, origem);
21.       }
22.       static void Main(string[] args)
23.       {
24.          Console.WriteLine("Torre de Hanói");
25.          // Suponhamos que queremos resolver o quebra-cabeça para 3
discos
26.          int numeroDeDiscos = 3;
27.          // O método é chamado com os pinos de origem (A), destino (C)
e auxiliar (B)
28.          MoverDiscos(numeroDeDiscos, 'A', 'C', 'B');
29.       }
30.    }
31. }
```

Figura 166 – Torre de Hanói: exemplo clássico do uso de recursão

A técnica recursiva funciona assim: em um dado quebra-cabeça, se quisermos mover n discos de uma haste origem para uma haste destino, podemos começar movendo n-1 discos para uma haste auxiliar, mover o disco n para a haste destino, e depois mover os n-1 discos da haste auxiliar para a haste destino.

O código começa com o método MoverDiscos (linha 7), que é a parte principal da implementação e é recursivo. Para mover apenas um disco (caso-base da recursão), fazemos isso diretamente. Se tivermos mais de um disco, primeiro movemos quantidade –1 discos para o pino auxiliar, depois movemos o disco restante para o destino e, finalmente, movemos os quantidade –1 discos do pino auxiliar para o destino.

A função Main inicia o processo e define o número de discos e os pinos (linhas 26 e 28, respectivamente). A saída do programa (figura 167) detalha as etapas para resolver o quebra-cabeça, indicando de qual (e para qual) pino cada disco deve ser movido. Ao usar a recursão, o programa decompõe o problema em problemas menores, tornando a solução da Torre de Hanói mais gerenciável e direta.

```
Torre de Hanói
Mova o disco 1 de A para C
Mova o disco 2 de A para B
Mova o disco 1 de C para B
Mova o disco 3 de A para C
Mova o disco 1 de B para A
Mova o disco 2 de B para C
Mova o disco 1 de A para C
```

Figura 167 – Torre de Hanói: resultado da execução no console

Uma **fila** pode ser comparada a uma fila comum, de pessoas esperando em um banco, por exemplo. A primeira pessoa que entra na fila é a primeira a ser atendida e a sair dela – comportamento conhecido como *first in, first out* (Fifo). Assim, em uma fila temos operações principais, como **enqueue** – que adiciona um elemento ao final dela – e **dequeue** – que remove o elemento do início.

Filas envolvem gestão de tarefas em sequência, como em sistemas de impressão, onde o primeiro documento enviado para a impressora é aquele impresso antes. Se vários usuários enviarem documentos para uma impressora ao mesmo tempo, ela os enfileira e os imprime em ordem, baseando-se no Fifo. Por exemplo, quando um e-mail é enviado, ele não é transmitido imediatamente para o destinatário; em vez disso, é colocado em uma fila de mensagens a enviar. O servidor de e-mail processa essa fila, enviando uma mensagem de cada vez, na ordem em que foram recebidas.

Outro exemplo clássico de filas no mundo real é o gerenciamento de tarefas em impressoras. Adicionalmente, centrais de atendimento ou sistemas de suporte utilizam filas. Quando ligamos para um serviço de atendimento ao cliente e ouvimos a mensagem "Você é o terceiro na fila", isso indica que há um sistema de fila gerenciando as chamadas.

Tanto pilhas quanto filas são facilmente implementadas, respectivamente, através das classes Stack e Queue, disponíveis na biblioteca-padrão .NET. Elas oferecem métodos e propriedades que facilitam a manipulação e o gerenciamento de dados de acordo com as características inerentes a cada estrutura.

```
1.  using System;
2.  using System.Collections.Generic;
3.  namespace PilhaMitologiaGrega
4.  {
5.     class Program
6.     {
7.        static void Main(string[] args)
8.        {
9.           // Criando uma pilha para armazenar as entidades mitológicas
10.          Stack<string> pilhaDeEntidades = new Stack<string>();
11.          // Adicionando entidades à pilha (push)
12.          pilhaDeEntidades.Push("Zeus");
13.          pilhaDeEntidades.Push("Athena");
14.          pilhaDeEntidades.Push("Poseidon");
15.          Console.WriteLine("Entidades adicionadas à pilha:");
16.          foreach (var entidade in pilhaDeEntidades)
17.          {
18.             Console.WriteLine(entidade);
19.          }
20.          // Suponhamos que Poseidon precisa retornar ao mar, então o
removemos (pop) do topo da pilha
21.          string entidadeRemovida = pilhaDeEntidades.Pop();
22.          Console.WriteLine($"\nEntidade que retornou ao seu reino:
{entidadeRemovida}");
23.          Console.WriteLine("\nPilha de entidades após remoção:");
24.          foreach (var entidade in pilhaDeEntidades)
25.          {
26.             Console.WriteLine(entidade);
27.          }
28.       }
29.    }
30. }
```

Figura 168 – Stack: exemplo de utilização

No exemplo usamos a estrutura Stack para gerenciar uma pilha de entidades da mitologia grega. Criamos uma pilha vazia (linha 10), e então três entidades mitológicas foram adicionadas à pilha com o método Push (linhas 12, 13 e 14); a última entidade adicionada, Poseidon, está no topo. Em seguida, exibimos essas entidades no console, usando um loop foreach.

Como parte da narrativa, suponhamos que Poseidon precise retornar ao seu reino, o mar. Para simular isso, utilizamos o método Pop para removê-lo da pilha (linha 21). Posteriormente, exibimos a entidade removida e mostramos a pilha após essa remoção. A estrutura de pilha permite simular o agrupamento e a retirada de entidades de um conjunto, seguindo o princípio Lifo.

```
1.  using System;
2.  using System.Collections.Generic;
3.  namespace FilaMitologiaGrega
4.  { class Program
5.     {
6.        static void Main(string[] args)
7.        { // Criando uma fila para armazenar as entidades mitológicas que
desejam falar ao oráculo
8.          Queue<string> filaOraculo = new Queue<string>();
9.          // Adicionando entidades à fila (enqueue)
10.         filaOraculo.Enqueue("Hércules");
11.         filaOraculo.Enqueue("Perseu");
12.         filaOraculo.Enqueue("Aquiles");
13.         Console.WriteLine("Entidades na fila para falar com o
oráculo:");
14.         foreach (var entidade in filaOraculo)
15.         { Console.WriteLine(entidade); }
16.         // A primeira entidade é atendida pelo oráculo, sendo removida
(dequeue) da fila
17.         string entidadeAtendida = filaOraculo.Dequeue();
18.         Console.WriteLine($"\nEntidade que falou com o oráculo:
{entidadeAtendida}");
19.         Console.WriteLine("\nFila após o atendimento do oráculo:");
20.         foreach (var entidade in filaOraculo)
21.         { Console.WriteLine(entidade); }
22.       }
23.    }
24. }
```

Figura 169 – Queue: exemplo de utilização

Usamos a estrutura Queue para simular uma fila de entidades mitológicas que desejam consultar o oráculo. A história começa com uma fila vazia (linha 8), e em seguida três entidades mitológicas – Hércules, Perseu e Aquiles – são adicionadas à fila usando o método Enqueue (linhas 10, 11 e 12). Nesse momento Hércules é o primeiro da fila, já que ele foi o primeiro a ser adicionado; depois exibimos as entidades na fila, mostrando a ordem em que elas aguardam para falar com o oráculo.

Dando sequência à narrativa, o oráculo está pronto para atender a primeira entidade, e assim Hércules, depois de atendido, é removido da fila com o método Dequeue (linha 17). Então mostramos quem foi atendido e exibimos a fila atualizada, que agora tem Perseu como o primeiro da fila. O código demonstra de maneira prática o funcionamento da estrutura de fila em C#, operando pelo princípio Fifo. A fila, como representada no código-fonte, garante que o primeiro a entrar é também o primeiro a ser atendido.

Pilhas e filas são mais do que meras abstrações da ciência da computação; estão incorporadas em muitos sistemas e aplicativos que usamos no dia a dia, embora possamos não percebê-las diretamente. Entender o funcionamento dessas estruturas é crucial para qualquer desenvolvedor, pois fornecem mecanismos para gerenciar dados de maneira ordenada e previsível, atendendo a diferentes necessidades e cenários no mundo da programação.

8.3 Dicionários e conjuntos – tabelas de hash

Dicionários e conjuntos são estruturas de dados que armazenam elementos de forma eficiente, permitindo operações rápidas de busca, inserção e remoção; essas estruturas são implementadas usando uma técnica chamada **tabelas de hash**. Um dicionário, especificamente, armazena pares de chave e valor, que em um dicionário são únicas, e cada chave está associada a um valor específico. Isso permite que os usuários acessem um valor rapidamente, fornecendo sua chave correspondente.

Por outro lado, um **conjunto**, ou set, armazena valores únicos, sem associações de chave-valor. Ambos, dicionários e conjuntos, aproveitam a técnica de tabelas de hash para alcançar eficiência. Essas tabelas trabalham convertendo a chave (ou valor, no caso de conjuntos) em um índice de array usando uma função hash. Esse índice determina onde o valor associado será armazenado na tabela. Devido a essa abordagem, o tempo necessário para encontrar um item em uma tabela não depende diretamente do número de elementos armazenados na estrutura; no entanto é importante observar que a eficiência de uma tabela depende de uma boa função hash e da capacidade de lidar com colisões (quando duas chaves diferentes resultam no mesmo índice).

Imagine que você esteja usando um serviço de streaming de música como o Spotify. Quando busca por uma música ou artista, o sistema precisa encontrar essa informação rapidamente em meio a milhões de músicas e artistas disponíveis. A busca eficiente em grandes volumes de dados pode ser viabilizada por dicionários, onde a chave pode ser o nome da música ou do artista, e o valor associado é a informação detalhada ou o local onde a música está armazenada.

Outro exemplo cotidiano são as redes sociais. Ao recebermos sugestões de "Pessoas que talvez conheça", o sistema, por trás das cenas, pode usar conjuntos para armazenar e processar rapidamente ID de usuários dos quais já somos amigos, garantindo que as sugestões não incluam pessoas que já fazem parte da nossa lista.

Em ambientes empresariais, sistemas de gestão de estoque podem usar dicionários para manter o controle de produtos. Cada produto pode ter um código de barras único que atua como chave, enquanto os detalhes e a quantidade em estoque do produto representam o valor associado. Já em segurança digital e sistemas de autenticação, as tabelas de hash são cruciais. Senhas, por exemplo, não devem ser armazenadas em formato puro; em vez disso, muitos sistemas armazenam uma versão hashed. Quando um usuário tenta fazer login, o sistema "hasheia" a senha inserida e a compara com a versão hashed armazenada. Isso torna a armazenagem de senhas mais segura e ainda aproveita a eficiência das operações de hash.

Existem classes específicas projetadas para implementar dicionários e conjuntos, proporcionando aos desenvolvedores as ferramentas necessárias para armazenar e gerenciar dados de maneira eficaz e eficiente. O C# oferece a classe Dictionary<TKey, TValue>, que faz parte do namespace System. Collections.Generic. Essa classe permite que os desenvolvedores armazenem pares de chave-valor; com ela é possível associar uma chave única a um valor específico, e essa associação garante que os dados sejam recuperados de maneira rápida, aproveitando as características subjacentes das tabelas de hash.

Esse dicionário é especialmente útil quando a busca rápida e a recuperação de dados são essenciais, como em sistemas de cache ou quando se deseja mapear identificadores a objetos complexos.

Por outro lado, quando se trata de conjuntos, o C# fornece a classe HashSet<T>, que também é parte do namespace System.Collections.Generic. A principal característica dos conjuntos é que eles armazenam itens únicos, ou seja, não permitem duplicatas, tornando-os ideais quando se deseja manter uma coleção de itens distintos, como uma lista de IDs de usuários únicos. A classe HashSet<T> garante que os itens sejam armazenados e acessados de forma eficiente, também aproveitando os princípios das tabelas de hash.

Ambas as classes, Dictionary<TKey, TValue> e HashSet<T>, são representantes claros de como o C# encapsula e oferece implementações de alto nível de estruturas de dados complexas, que permitem aos desenvolvedores se beneficiar da eficiência e eficácia das tabelas de hash, sem se aprofundar nos detalhes intrincados de sua implementação, permitindo que se concentrem em resolver problemas e construir aplicações robustas.

Observe o código-fonte da figura 170. Nele começamos importando o namespace System. Collections.Generic, que dá acesso à classe Dictionary<TKey, TValue>. Criamos um dicionário chamado deusesGregos que armazena nomes de deuses como chaves e suas descrições como valores. Adicionados alguns pares de chave-valor, tentamos recuperar e exibir a descrição de um deus específico, no caso, Athena.

Dicionários em C# proporcionam uma maneira eficiente de armazenar e acessar pares de chave-valor. No exemplo os deuses da mitologia grega são usados como chaves, e suas descrições como os respectivos valores. Ao adicionar deuses ao dicionário, podemos facilmente recuperar informações específicas usando a função TryGetValue. A escolha de Athena no exemplo foi meramente ilustrativa; esse código poderia ser facilmente estendido para permitir interações mais complexas, como consultar qualquer deus ou adicionar novos deuses e descrições ao dicionário.

```
1.  using System;
2.  using System.Collections.Generic;
3.
4.  class Program
5.  {
6.     static void Main()
7.     {
8.        // Criando um dicionário para armazenar os deuses e suas
descrições
9.        Dictionary<string, string> deusesGregos = new Dictionary<string,
string>();
10.
11.       // Adicionando alguns deuses e suas descrições ao dicionário
12.       deusesGregos.Add("Zeus", "Deus do céu e rei dos deuses.");
13.       deusesGregos.Add("Athena", "Deusa da sabedoria, da coragem e da
inspiração.");
14.       deusesGregos.Add("Poseidon", "Deus do mar, dos terremotos e dos
cavalos.");
15.
16.       // Consultando e exibindo a descrição de um deus específico
17.       string deusEscolhido = "Athena";
18.       if (deusesGregos.TryGetValue(deusEscolhido, out string
descricao))
19.       {
20.          Console.WriteLine($"{deusEscolhido}: {descricao}");
21.       }
22.       else
23.       {
24.          Console.WriteLine($"Desculpe, {deusEscolhido} não foi
encontrado no nosso dicionário.");
25.       }
26.    }
27. }
```

Figura 170 – Dictionary: exemplo de utilização

Na figura 171, utilizando o namespace System.Collections.Generic, temos acesso à classe HashSet<T>. Criamos um conjunto chamado monstrosGregos para armazenar nomes de monstros da mitologia grega. Adicionados alguns monstros, tentamos adicionar novamente a Medusa, demonstrando que o HashSet previne duplicatas automaticamente.

Por fim verificamos se um monstro, no caso Cérbero, está presente no conjunto. O HashSet<T> em C# é uma estrutura de dados que permite armazenar uma coleção de elementos únicos, evitando repetições. No exemplo a mitologia grega é usada como contexto para armazenar monstros em um conjunto, e o ato de tentar adicionar a Medusa duas vezes ilustra a propriedade fundamental de um conjunto: a exclusividade de seus elementos. Ao verificar a presença do Cérbero, o código demonstra como consultar elementos em um conjunto.

```
1.  using System;
2.  using System.Collections.Generic;
3.
4.  class Program
5.  {
6.     static void Main()
7.     {
8.        // Criando um conjunto (HashSet) para armazenar monstros da
mitologia grega
9.        HashSet<string> monstrosGregos = new HashSet<string>();
10.
11.       // Adicionando alguns monstros ao conjunto
12.       monstrosGregos.Add("Minotauro");
13.       monstrosGregos.Add("Medusa");
14.       monstrosGregos.Add("Hidra");
15.
16.       // Tentando adicionar um monstro repetido ao conjunto
17.       if (!monstrosGregos.Add("Medusa"))
18.       {
19.          Console.WriteLine("Medusa já está no conjunto de monstros!");
20.       }
21.
22.       // Verificando e exibindo se um monstro específico está no conjunto
23.       string monstroEscolhido = "Cérbero";
24.       if (monstrosGregos.Contains(monstroEscolhido))
25.       {
26.          Console.WriteLine($"{monstroEscolhido} está no conjunto de
monstros!");
27.       }
28.       else
29.       {
30.          Console.WriteLine($"{monstroEscolhido} não foi encontrado no
conjunto.");
31.       }
32.    }
33. }
```

Figura 171 – HashSet: exemplo de utilização

Dicionários e conjuntos são, portanto, ferramentas poderosas no arsenal de um desenvolvedor C#, pois oferecem operações rápidas e eficientes e são particularmente úteis em cenários onde a busca rápida de itens é essencial. Ao compreender como as tabelas de hash funcionam, desenvolvedores podem fazer uso mais informado e otimizado dessas estruturas.

8.4 Árvores – binária, busca binária, autobalanceada

Dentro do domínio das ciências da computação e das estruturas de dados, **árvores** são uma das abordagens mais interessantes para organizar informações de forma hierárquica. Em C#, assim como em muitas outras linguagens, árvores têm uma presença marcante, especialmente quando se trata de armazenamento e recuperação eficientes de dados. Imagine uma árvore virada de cabeça para baixo, com a raiz no topo e os galhos se espalhando para baixo; esse é o arquétipo de uma árvore em ciências da computação.

Árvore binária é uma delas, onde cada nó tem no máximo dois filhos, muitas vezes denominados subárvore esquerda e subárvore direita. Uma característica importante de algumas árvores binárias é a forma como são organizadas. Quando se tem uma **árvore de busca binária**, por exemplo, os valores são dispostos de tal forma que, para qualquer nó dado, os valores à esquerda são menores que o valor do nó, e os valores à direita são maiores. Essa disposição permite uma busca eficiente, pois elimina metade das opções possíveis a cada passo.

No entanto, uma preocupação ao usar árvore de busca binária é seu equilíbrio. Se os dados forem inseridos em uma ordem particular, ela pode ficar muito inclinada, assemelhando-se mais a uma lista vinculada. Esta situação elimina os benefícios de tempo de busca da estrutura de árvore. É aqui que entram as **árvores autobalanceadas**. Estas são árvores de busca binária que se reorganizam durante as operações de inserção e remoção para garantir que a árvore permaneça equilibrada, otimizando assim o tempo de busca.

Ao pensarmos na vida cotidiana e em todos os sistemas digitais que a permeiam, muitos deles têm, em seu núcleo, estruturas de dados fundamentais, como as árvores. As árvores, em especial as binárias de busca e suas variantes autobalanceadas, desempenham um papel crucial em muitos cenários que talvez não percebamos à primeira vista. Por exemplo, quando usamos um banco de dados relacional, muitas vezes as operações de busca, inserção e exclusão de dados são eficientemente gerenciadas por árvores binárias de busca ou suas variantes mais sofisticadas. Esses bancos precisam ser extremamente rápidos ao responder às consultas, e a estrutura organizada e equilibrada das árvores de busca binária é perfeitamente adequada para essa finalidade.

Pense em um bibliotecário que, em vez de procurar cada prateleira individualmente, sabe exatamente aonde ir para encontrar um livro, simplesmente seguindo a estrutura organizada. Além disso, sistemas de gerenciamento de arquivos em muitos sistemas operacionais usam árvores para organizar diretórios e subdiretórios. Essa organização hierárquica facilita a busca e o armazenamento de arquivos no disco.

A navegação por *global positioning system* (GPS), que muitos de nós usamos regularmente, também emprega árvores. Ao calcular o caminho mais rápido de um ponto A para um ponto B, muitos algoritmos de otimização trabalham com estruturas de dados baseadas em árvores para armazenar e processar caminhos possíveis. Até mesmo quando digitamos em uma barra de pesquisa de um mecanismo de busca, as sugestões que aparecem à medida que escrevemos podem estar sendo geradas a partir de uma estrutura de árvore chamada Trier. Embora não seja exatamente uma árvore binária, ainda é uma forma de árvore e um lembrete de como essas estruturas permeiam nossas interações diárias.

A biblioteca-padrão **não** fornece uma implementação direta dessas árvores, mas estruturas como SortedSet<T> e SortedDictionary<TKey, TValue> se baseiam em árvores e oferecem características de desempenho semelhantes. Essas classes encapsulam a complexidade das operações da árvore, permitindo que os desenvolvedores se beneficiem de suas vantagens sem se aprofundar nos detalhes de sua implementação.

No programa da figura 172, utilizamos a classe SortedSet<T>, a qual garante que os elementos sejam não apenas únicos, mas também ordenados. Aqui temos um conjunto de deuses gregos, e a ordenação-padrão, que é alfabética para strings, garante que eles sejam exibidos em sequência alfabética. Em seguida, recorremos à SortedDictionary<TKey, TValue> para mapear os deuses a seus domínios. O dicionário não apenas mantém os pares chave-valor, mas também assegura que as chaves estejam em ordem. Então, quando percorremos o dicionário e exibimos os domínios dos deuses, vemos essa ordenação em ação.

Note que no código da figura 172 **não** implementamos explicitamente uma árvore; em vez disso utilizamos as abstrações fornecidas pela biblioteca-padrão do .NET que, internamente, usam árvores para garantir ordenação e eficiência nas operações. As classes SortedSet<T> e SortedDictionary<TKey, TValue> do .NET se baseiam em árvores binárias de busca. Sobre serem autobalanceadas, o .NET usa árvores Red-Black para implementar essas estruturas. Red-Black é uma árvore binária de busca autobalanceada que permanece aproximadamente balanceada durante inserções e exclusões. Isso é fundamental para garantir que operações como busca, inserção e remoção permaneçam eficientes, mesmo quando a quantidade de elementos é grande.

Assim, ao utilizar o SortedSet<T> ou SortedDictionary<TKey, TValue> no .NET, o programador está indiretamente usando uma árvore binária de busca autobalanceada; mas vale destacar que essa implementação é abstraída, ou seja, detalhes internos como balanceamento são gerenciados pela própria estrutura, permitindo que o desenvolvedor se concentre apenas em operações de alto nível.

```
1.  using System;
2.  using System.Collections.Generic;
3.  class Program
4.  {
5.     static void Main()
6.     {
7.        // Usando SortedSet<T> para armazenar deuses gregos em ordem
alfabética.
8.        SortedSet<string> deusesGregos = new SortedSet<string>
9.        {
10.          "Apolo", "Ares", "Atena", "Deméter", "Dionísio", "Hades",
"Hera", "Hermes", "Poseidon", "Zeus"
11.       };
12.       Console.WriteLine("Deuses gregos em ordem alfabética:");
13.       foreach (var deus in deusesGregos)
14.       {
15.          Console.WriteLine(deus);
16.       }
17.       // Usando SortedDictionary<TKey, TValue> para relacionar deuses
gregos a seus domínios.
18.       SortedDictionary<string, string> dominiosDosDeuses = new
SortedDictionary<string, string>
19.       {
20.          {"Apolo", "Arte e música"},
21.          {"Ares", "Guerra"},
22.          {"Atena", "Sabedoria"},
23.          {"Deméter", "Agricultura"},
24.          {"Dionísio", "Vinho e festas"},
25.          {"Hades", "Submundo"},
26.          {"Hera", "Casamento"},
27.          {"Hermes", "Mensageiro dos deuses"},
28.          {"Poseidon", "Mares"},
29.          {"Zeus", "Céu e trovões"}
30.       };
31.       Console.WriteLine("\nDeuses gregos e seus domínios:");
32.       foreach (var par in dominiosDosDeuses)
33.       {
34.          Console.WriteLine($"{par.Key} é o deus(a) de {par.Value}.");
35.       }
36.    }
37. }
```

Figura 172 – SortedSet<T> e SortedDictionary<TKey, TValue>: exemplo de utilização

8.5 Heaps – binário, binomial, Fibonacci

Em computação, heaps desempenham papel fundamental em muitos algoritmos e aplicações. São um tipo especial de árvore binária (ou, em alguns casos, não binária) que segue determinadas propriedades. O **heap binário** é, talvez, o mais familiar de todos. Trata-se de uma árvore binária completa, onde cada nó tem um valor que respeita uma relação específica se comparado aos valores de seus nós filhos.

Existem dois tipos principais de heaps binários: max-heaps e min-heaps. Em um max-heap, o valor de cada nó é sempre maior ou igual aos valores de seus filhos, enquanto em um min-heap é sempre menor ou igual. Isso permite operações eficientes como inserção, exclusão e, mais notavelmente, extração do elemento máximo ou mínimo.

Já os **heaps binomiais** são uma extensão interessante da ideia dos heaps binários – uma coleção de árvores binomiais, onde cada árvore é um heap. A peculiaridade dos heaps binomiais é permitir operações como união de dois heaps de maneira eficiente, em um tipo de heap formado pela combinação de outros heaps binomiais menores, seguindo propriedades específicas que garantem sua eficiência.

Em uma abordagem ainda mais sofisticada, encontramos os **heaps de Fibonacci**, uma coleção de árvores que não necessariamente precisam ser binárias e que são muito hábeis em certas operações, como diminuir uma chave ou excluir um nó em tempo amortizado constante. Essa eficiência torna-as uma escolha atrativa para algoritmos que necessitam de operações prioritárias rápidas, como o algoritmo de Dijkstra para caminhos mais curtos (Dijkstra, 1959), uma abordagem fundamental na teoria dos grafos para encontrar o caminho mais curto entre dois nós em um grafo ponderado. Foi concebido pelo cientista da computação holandês Edsger W. Dijkstra.

Existem diversos usos de heaps no dia a dia. Quando múltiplos processos ou threads disputam acesso a recursos, como uma CPU, o sistema operacional precisa tomar decisões rápidas sobre quem vai na frente e quem espera. Heaps binários, por sua eficiência em operações de inserção e remoção, são frequentemente empregados nesses algoritmos para gerenciar a prioridade de tarefas. Quando fazemos uma busca, os mecanismos de pesquisa têm o desafio colossal de retornar os resultados mais relevantes dentre bilhões de páginas. Em algum momento esses mecanismos podem usar heaps para ajudar a identificar e ordenar esses resultados por relevância, efetuando operações de inserção e remoção à medida que novos candidatos a resultados relevantes são avaliados.

Entrando no universo das redes, temos o cenário das redes de computadores e comunicações. Algoritmos de caminho mais curto, como o Dijkstra – essenciais para determinar a rota mais rápida para transmitir dados em uma rede –, aproveitam-se da estrutura heap para acelerar o processo de encontrar o próximo nó a ser explorado. E em aplicações financeiras – como sistemas de negociação que precisam rapidamente casar ordens de compra e venda – um heap pode manter as ordens organizadas por preço, permitindo que as operações sejam casadas eficientemente.

Embora a linguagem não forneça implementações-padrão para todas essas estruturas de dados na biblioteca .NET, os conceitos permanecem relevantes. Desenvolvedores frequentemente recorrem

a bibliotecas externas ou implementam suas próprias versões desses heaps quando necessitam de propriedades e operações especializadas.

8.6 Grafos – Kruskal, Prim e Dijkstra

Grafos são estruturas de dados que podem representar relações binárias entre um conjunto de objetos. São formados por vértices, também conhecidos como nós, conectados por arestas ou arcos. São amplamente usados para resolver problemas e modelar sistemas em campos variados, como redes de computadores, sistemas de transporte e redes sociais. Em essência, são instrumentos matemáticos extremamente valiosos e versáteis, aplicáveis em diversas situações cotidianas e profissionais, servindo como base para modelar e resolver uma variedade de problemas práticos em diferentes domínios. Frequentemente representam relações entre objetos ou entidades, proporcionando uma representação visual e estrutural de sistemas complexos.

Um exemplo proeminente de aplicação de grafos acontece nas redes de transporte público, onde estações ou paradas são representadas como vértices, e as rotas de transporte, sejam linhas de ônibus, metrô ou trem, são representadas como arestas. A modelagem dessas redes como grafos permite analisar e otimizar o fluxo de transporte, facilitando o planejamento de rotas eficientes e a identificação de gargalos na rede, contribuindo para melhorias significativas na mobilidade urbana.

Grafos são indispensáveis no campo das telecomunicações e redes de computadores. Dispositivos de rede – como roteadores e switches – podem ser representados como vértices, enquanto as conexões ou links entre esses dispositivos são as arestas. O uso de teoria dos grafos nesse contexto possibilita projetar e analisar topologias de rede, identificar caminhos mais curtos ou mais eficientes entre dispositivos e detectar e resolver problemas como loops e pontos de falha na rede.

Também desempenham papel crucial na modelagem e análise de redes sociais. Indivíduos dentro de uma rede social são representados como vértices, e suas interações, amizades ou segmentos são representados como arestas, o que permite analisar padrões de interação e identificar comunidades, influenciadores e indivíduos isolados. Analisando grafos é possível obter insights valiosos sobre a estrutura e a dinâmica das redes sociais, com implicações significativas para o marketing, a sociologia e a psicologia.

Algoritmo de Kruskal é uma técnica que encontra a árvore geradora mínima em um grafo (Kruskal, 1956), ou seja, um subconjunto do grafo que conecta todos os vértices com o menor custo total. Ele começa tratando cada vértice como componente separado e, iterativamente, seleciona a aresta de menor peso que conecta dois componentes distintos, continuando esse processo até que todos os vértices estejam conectados.

O **algoritmo de Prim**, por outro lado, também encontra a árvore geradora mínima em um grafo, mas adota uma abordagem diferente do Kruskal; ele começa a partir de um vértice arbitrário e, a cada passo, adiciona ao conjunto a aresta de menor peso que conecta um vértice dentro do conjunto a um vértice fora dele, continuando o processo até que todos os vértices estejam no conjunto.

Por fim, o **algoritmo de Dijkstra** encontra o caminho mais curto entre dois vértices em um grafo ponderado, onde arestas têm pesos associados a elas, que podem representar, por exemplo, distâncias ou custos. Esse algoritmo trabalha de maneira incremental, começando do vértice de origem e expandindo-se para os vértices vizinhos, atualizando as distâncias conhecidas até o momento, até alcançar o vértice de destino.

Um grafo pode ser direcionado ou não direcionado, ponderado ou não ponderado. Em um grafo direcionado, cada aresta tem uma direção, indo de um vértice origem para um vértice destino. Em um grafo não direcionado, as arestas não têm direção. Grafos ponderados têm valor ou peso associado a cada aresta, representando, por exemplo, a distância entre dois pontos ou o custo para percorrer uma conexão. Grafos não ponderados não têm pesos associados às suas arestas.

Na linguagem de programação C#, grafos podem ser representados e manipulados através da criação de classes e objetos. Uma abordagem comum é criar uma classe para representar o grafo, uma classe para os vértices e, se necessário, uma classe para as arestas. Estruturas de dados, como listas e dicionários, podem armazenar vértices e arestas, e manter informações sobre as relações entre eles.

Ao implementar grafos em C#, a orientação a objetos da linguagem permite que os desenvolvedores encapsulem os detalhes da implementação e exponham interfaces claras e coesas. Além disso, métodos e propriedades em C# facilitam a manipulação de grafos, permitindo que operações como adição de vértices, adição de arestas e busca de caminhos sejam executadas de maneira intuitiva e segura.

Resumo

Desenvolver soluções eficazes em C# implica um entendimento abrangente de diversas funcionalidades e conceitos avançados da linguagem, que se interconectam para desenvolver sistemas robustos e eficientes. Expressões lambda e LINQ são exemplares nessa questão, visto que possibilitam construir consultas e operações de manipulação de dados mais concisas e expressivas, permitindo que os desenvolvedores interajam com coleções de dados, como dicionários e listas, de maneira mais fluida e legível.

Na manipulação e no armazenamento de dados é indispensável entender estruturas como pilhas, filas, árvores, heaps e grafos, cada qual com suas propriedades e usos específicos, como ordenação e busca. O uso adequado dessas estruturas impacta diretamente a performance e a eficiência dos algoritmos, enquanto o emprego de dicionários e conjuntos viabiliza o armazenamento e a recuperação eficaz de dados através da associação entre chaves e valores.

Já no contexto de execução e performance, conceitos como threads, tasks e funções assíncronas são primordiais para desenvolver aplicações que otimizam o uso de recursos do sistema e melhoram a responsividade, ao permitirem a execução concorrente de múltiplas tarefas. Além disso, entender os mecanismos de garbage collection é igualmente crucial para garantir a gestão eficiente da memória, liberando recursos em desuso e prevenindo vazamentos de memória que podem degradar a performance do sistema.

Também é essencial incorporar práticas de debugging e tracing para identificar e corrigir falhas no código, oferecendo insights valiosos sobre o comportamento do sistema em tempo de execução e possibilitando detectar problemas que podem não ser evidentes durante o desenvolvimento.

Cada um desses conceitos e técnicas não apenas enriquece o arsenal dos desenvolvedores de C#, como também os habilita a elaborar soluções mais sofisticadas, resilientes e eficientes, respondendo de forma mais precisa demandas e desafios que surgem no cenário da tecnologia da informação contemporânea.

Portanto, a proficiência em tais conceitos é vital para desenvolver software em C#, contribuindo diretamente para elevar a qualidade e robustez das soluções desenvolvidas.

Exercícios

Questão 1. (Fundatec 2023, adaptada) A ordenação de vetores e de listas é uma atividade crucial na programação. Ao longo dos anos, diversos algoritmos foram desenvolvidos com o intuito de realizar essa tarefa. Embora não seja considerado um algoritmo de grande eficiência, o Bubble Sort é um dos algoritmos de ordenação mais conhecidos, devido, principalmente, à sua simplicidade.

Nesse cenário, considere que um programador queira ordenar a lista [5, 4, 3]. Qual alternativa representa todas as situações dessa lista, até a sua completa ordenação, utilizando o método Bubble Sort?

A) [5, 4, 3], [4, 3, 5], [3, 4, 5], [3, 4, 5], [3, 4, 5]

B) [5, 4, 3], [4, 5, 3], [4, 5, 3], [3, 4, 5], [3, 4, 5]

C) [5, 4, 3], [4, 5, 3], [3, 4, 5]

D) [5, 4, 3], [4, 5, 3], [4, 3, 5], [3, 4, 5], [3, 4, 5]

E) [5, 4, 3], [4, 3, 5], [3, 4, 5]

Resposta correta: alternativa D.

Análise da questão

O algoritmo de ordenação Bubble Sort atua comparando pares de elementos adjacentes na lista, trocando-os se estiverem na ordem errada. O algoritmo repete esse processo até que a lista esteja completamente ordenada.

As etapas de ordenação da lista do enunciado, de acordo com a sequência do Bubble Sort, são elencadas a seguir, lembrando que a situação inicial da lista é **[5, 4, 3]**.

1. Na primeira varredura da lista, inicialmente é feita a comparação entre o primeiro par de elementos adjacentes, que são os elementos 5 e 4. Como 5 é maior do que 4, há a troca desses elementos, o que leva a lista à situação **[4, 5, 3]**.

2. Em seguida, o algoritmo avança para comparar o próximo par de elementos, que são os elementos 5 e 3. Como 5 é maior do que 3, há a troca desses elementos, o que leva a lista à situação **[4, 3, 5]**. A primeira varredura da lista está completa.

3. Na segunda varredura, partimos da situação [4, 3, 5]. Inicialmente é feita a comparação entre o primeiro par de elementos adjacentes, que são os elementos 4 e 3. Como 4 é maior do que 3, há a troca desses elementos, o que leva a lista à situação **[3, 4, 5]**.

4. Em seguida, o algoritmo avança para comparar o próximo par de elementos, que são os elementos 4 e 5. Como 4 não é maior do que 5, não há a troca desses elementos, o que mantém a lista na situação **[3, 4, 5]**. A segunda varredura está completa.

5. É feita uma última varredura, que permite verificar que a lista já está completamente ordenada.

Questão 2. (Instituto AOCP 2019, adaptada) Entender o funcionamento de estruturas de dados elementares é essencial para desenvolvedores, pois elas fornecem mecanismos para gerenciar dados de maneira ordenada e previsível, atendendo a diferentes necessidades e cenários.

Sobre as estruturas de dados pilha e fila, avalie as afirmativas.

I – Em uma pilha é implementada uma política *last in*, *first out* (Lifo).

II – Nas filas a política implementada é a *first in*, *first out* (Fifo).

III – O atributo "topo" de uma pilha indexa o elemento mais recentemente inserido.

É correto o que se afirma em:

A) I, apenas.

B) II, apenas.

C) I e II, apenas.

D) II e III, apenas.

E) I, II e III.

Resposta correta: alternativa E.

Análise das afirmativas

I – Afirmativa correta.

Justificativa: uma pilha segue a política Lifo. Isso significa que o último elemento inserido será o primeiro a ser retirado. Essa estrutura de dados pode ser comparada a uma pilha de pratos, em que o último prato a ser colocado na estrutura ocupa o topo e será o primeiro a ser retirado.

II – Afirmativa correta.

Justificativa: uma fila segue a política Fifo. Isso significa que o primeiro elemento inserido será o primeiro a ser retirado. Essa estrutura de dados pode ser comparada a uma fila de clientes esperando por

atendimento em um banco. O primeiro cliente a chegar ocupará o lugar inicial da fila e será o primeiro cliente a ser atendido.

III – Afirmativa correta.

Justificativa: o atributo "topo" de uma pilha refere-se ao elemento mais recentemente inserido, que será o próximo a ser removido.

REFERÊNCIAS

Textuais

ALBAHARI, J. *C# 9.0 in a nutshell*: the definitive reference. Sebastopol: O'Reilly Media, 2021.

ASHBY, W. R. *An introduction to cybernetics*. London: Chapman & Hall, 1956.

BOOCH, G.; RUMBAUGH, J.; JACOBSON, I. *The unified modeling language user guide*. 2. ed. Reading: Addison-Wesley Professional, 2005.

BOX, D. *Essential.NET*. Volume I: the common language runtime. Boston: Addison-Wesley, 2002.

CAMÕES, L. *Rimas*. 1. ed. Coimbra: Biblioteca Geral da Universidade de Coimbra, 1953. v. 1598.

CLEARY, S. *Concurrency in C# cookbook*: asynchronous, parallel, and multithreaded programming. Sebastopol: O'Reilly Media, 2019.

CORMEN, T. H. *et al. Algoritmos*: teoria e prática. Rio de Janeiro: Elsevier, 2002.

DAHL, O.-J.; NYGAARD, K. SIMULA: An ALGOL-based simulation language. *Communications of the ACM*, p. 671-678, 9 set. 1966.

DIJKSTRA, E. W. A note on two problems in connexion with graphs. *Numerische Mathematik*, v. 1, p. 269-271, 1959.

DIJKSTRA, E. W. Go to statement considered harmful. *Communications of the ACM*, p. 147-148, 1968.

DIJKSTRA, E. W. Programming as a discipline of mathematical nature. *American Mathematical Monthly*, p. 608-612, 1979.

DOCUMENTAÇÃO do C#. *Learn Microsoft*, 26 maio 2017. Disponível em: https://tinyurl.com/yeypc4ww. Acesso em: 23 out. 2023.

FEATHERS, M. C. *Trabalho eficaz com código legado*. Porto Alegre: Bookman, 2013.

FONSECA FILHO, C. *História da computação*: o caminho do pensamento e da tecnologia. Porto Alegre: EDIPUCRS, 2007.

FREEMAN, A. *Pro ASP.NET Core 3*: develop cloud-ready web applications using MVC 3, blazor, and razor pages. London: Apress, 2019.

GAMMA, E. *et al. Design patterns*: elements of reusable object-oriented software. Boston: Addison-Wesley Professional, 1994.

GAMMA, E. *et al. Padrões de projeto*: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000.

GOGOL, N. *O nariz*. São Paulo: Cosac & Naify, 2008.

HEJLSBERG, A.; TORGERSEN, M. *The C# programming language*. Boston: Addison-Wesley, 2010.

HUSKEY, V. R.; HUSKEY, H. D. Lady Lovelace and Charles Babbage. *Annals of the History of Computing*, v. 2, p. 384, 1980.

JARGAS, A. *Expressões regulares*: uma abordagem divertida. São Paulo: Novatec, 2016.

KAY, A. Microelectronics and the personal computer. *Scientific American*, v. 237, n. 3, p. 230-244, 1977.

KNUTH, D. E. *The art of computer programming*: volume 3 – sorting and searching. Reading: Addison-Wesley, 1998.

KRUSKAL, J. B. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, v. 7, p. 48-50, 1956.

LISKOV, B.; WING, M. *Family values*: a behavioral notion of subtyping. Pittsburgh: Carnegie Mellon University, 1993.

MACHADO, A. Retrato (de Campos de Castilla). *Cuadernos Hispanoamericanos*, México, v. 11, p. 241-242, 1949.

MARTIN, R.; MARTIN, M. *Princípios, padrões e práticas ágeis em C#*. Porto Alegre: Bookman, 2011.

MCCARTHY, J. Recursive functions of symbolic expressions and their computation by machine, part I. *Communications of the ACM*, p. 184-195, 4 mar. 1960.

METZGAR, D. .NET *Core in Action*. Nova York: Manning Publications, 2018.

MEYER, B. The power of abstraction, reuse, and simplicity: an object-oriented library for event-driven design. *ACM Sigsoft Software Engineering Notes*, p. 39-46, 5 jul. 1982.

PARNAS, D. L. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, v. 12, p. 1053-1058, 1972.

PRECHELT, L. An empirical comparison of seven programming languages. *Computer*, v. 33, n. 10, p. 23-29, out. 2000.

PRIM, R. C. Shortest connection networks and some generalizations. *The Bell System Technical Journal*, v. 36, p. 1389-1401, 1957. ISSN 6.

REENSKAUG, T. The model-view-controller (MVC), its past and present. *Computer Science*, 2003.

RICHTER, J. *CLR via C#*. [S.l.]: Microsoft Press, 2012.

RUMBAUGH, J.; JACOBSON, I.; BOOCH, G. *The unified modeling language reference manual*. 2. ed. Boston: Addison-Wesley, 2004.

SAMMET, J. E. The early history of Cobol. *History of Programming Languages*, p. 199-243, 1978.

SEBESTA, R. W. *Conceitos de linguagens de programação*. 9. ed. Porto Alegre: Bookman, 2010. 792 p.

SEDGEWICK, R. *Algoritmos em C*. Porto Alegre: Bookman, 2013.

STEELE, G. L. Macaroni is better than spaghetti. *In:* PROCEEDINGS OF THE 1977 SYMPOSIUM ON ARTIFICIAL INTELLIGENCE AND PROGRAMMING LANGUAGES, 1., Nova York. *Proceedings* [...]. Nova York: Association for Computing Machinery, 1977. p. 60-66.

STRINGS and string literals. *Learn Microsoft*, 27 maio 2023. Disponível em: https://tinyurl.com/3633wrt2. Acesso em: 30 out. 2023.

TROELSEN, A.; JAPIKSE, P. *Pro C# 9 with.NET 5*. Nova York: Apress Berkeley, 2021.

WILKES, M. V. The EDSAC – an electronic calculating machine. *Journal of Scientific Instruments*, p. 385-391, 1949.

ZIVIANI, N. *Projeto de algoritmos*: com implementações em Pascal e C. São Paulo: Cengage Learning, 2004.

UNIP

UNIVERSIDADE PAULISTA

Informações:
www.sepi.unip.br ou 0800 010 9000